



SHIROMANI GURUDWARA PARBHANDHAK COMMITTEE'S
GURU NANAK KHALSA COLLEGE OF ARTS, SCIENCE & COMMERCE
(Autonomous)

(Affiliated to University of Mumbai and Recognized by Govt. of Maharashtra)

Accredited by NAAC with 'A' Grade with a CGPA of 3.54

**NGS – Precision Medicine Toolkit:
A Comprehensive User – Friendly GUI for
Next-Generation Sequencing (NGS) with Precision
Medicine Applications**

(M. Sc. Bioinformatics – Thesis Project)

**THESIS SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS
FOR THE DEGREE OF**

MASTER OF SCIENCE IN BIOINFORMATICS

SUBMITTED BY

Ms. Prarthi Hrishit Kothari
(Roll No.: 115)

UNDER THE GUIDANCE OF

Prof. Sermarani Nadar (Faculty, Department of Bioinformatics)
Dr. Gursimran Kaur Uppal (Head of Department, Department of Bioinformatics)

SUBMITTED TO

Department of Bioinformatics,
Guru Nanak Khalsa College of Arts, Science & Commerce (Autonomous),
Matunga (East), Mumbai – 400019

MARCH-2025



SHIROMANI GURUDWARA PARBHANDHAK COMMITTEE'S
GURU NANAK KHALSA COLLEGE OF ARTS, SCIENCE & COMMERCE
(Autonomous)

(Affiliated to University of Mumbai and Recognized by Govt. of Maharashtra)

Accredited by NAAC with 'A' Grade with a CGPA of 3.54

**NGS – Precision Medicine Toolkit:
A Comprehensive User – Friendly GUI for
Next-Generation Sequencing (NGS) with Precision
Medicine Applications**

(M. Sc. Bioinformatics – Thesis Project)

**THESIS SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS
FOR THE DEGREE OF**

MASTER OF SCIENCE IN BIOINFORMATICS

SUBMITTED BY

Ms. Prarthi Hrishit Kothari
(Roll No.: 115)

UNDER THE GUIDANCE OF

Prof. Sermarani Nadar (Faculty, Department of Bioinformatics)
Dr. Gursimran Kaur Uppal (Head of Department, Department of Bioinformatics)

SUBMITTED TO

Department of Bioinformatics,
Guru Nanak Khalsa College of Arts, Science & Commerce (Autonomous),
Matunga (East), Mumbai – 400019

MARCH-2025



SHIROMANI GURUDWARA PARBHANDHAK COMMITTEE'S
GURU NANAK KHALSA COLLEGE OF ARTS, SCIENCE & COMMERCE
(Autonomous)

(Affiliated to University of Mumbai and Recognized by Govt. of Maharashtra)

Accredited by NAAC with 'A' Grade with a CGPA of 3.54

DEPARTMENT OF BIOINFORMATICS

CERTIFICATE

Exam Seat No.: 115

This is to certify that Ms. Prarthi Hrishit Kothari (Roll No.: 115), student of M. Sc. Bioinformatics (Semester IV), at the Department of Bioinformatics, Guru Nanak Khalsa College of Arts, Science & Commerce (Autonomous), Mumbai – 400019, has submitted the thesis work titled, 'NGS – Precision Medicine Toolkit: A Comprehensive User – Friendly GUI for Next – Generation Sequencing (NGS) With Precision Medicine Applications' for the fulfilment of the Master of Science (M. Sc.) Degree in Bioinformatics, during the Academic year 2024-2025.

Professor – in – Charge
(Prof. Sermarani Nadar)

Head of Department
(Dr. Gursimran Kaur Uppal)

Examiner

Stamp of Institution

DECLARATION

I hereby declare that the project entitled. 'NGS – Precision Medicine Toolkit: A Comprehensive User – Friendly GUI for Next – Generation Sequencing (NGS) With Precision Medicine Applications', submitted by Ms. Prarthi Hrishit Kothari to the Department of Bioinformatics, Guru Nanak Khalsa College of Arts, Science & Commerce, Matunga, Mumbai, affiliated with the University of Mumbai is an original work done and submitted for the partial fulfilment of the degree of Master of Science in Bioinformatics, University of Mumbai.

This work is done by me under the guidance of Prof. Sermarani Nadar, Professor-in-charge, Master of Science in Bioinformatics (Part – II), Department of Bioinformatics, Guru Nanak Khalsa College of Arts, Science & Commerce, Matunga, Mumbai – 400019.

Student's signature
(Ms. Prarthi Hrishit Kothari)

Place:

Date:

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have supported me throughout the journey of my research project titled ‘NGS – Precision Medicine Toolkit: A Comprehensive User – Friendly GUI for Next – Generation Sequencing (NGS) with Precision Medicine Applications’.

Firstly, I extend my sincere thanks to Dr. Ratna Sharma (The Principal, of Guru Nanak Khalsa College of Arts, Science & Commerce (Autonomous)) and Dr. Gursimran Kaur Uppal (Head of Department of Bioinformatics), for fostering an environment that encourages exploration and creativity. Your leadership and vision have created a space where students can thrive and pursue their academic interests with passion.

Next, I would like to acknowledge the unwavering support of my guide, Prof. Sermarani Nadar, whose invaluable guidance, expertise, and encouragement have been instrumental in shaping this project. Your insights into the complexities of the field have not only enriched my understanding but have also inspired me to explore innovative solutions. Your dedication to teaching and your willingness to share your knowledge have provided me with a solid foundation in both theoretical and practical aspects of this research. I would also like to extend my heartfelt gratitude to Prof. Aparna Patil Kose, whose knowledge and support were instrumental in my academic growth.

I am also grateful to my peers and colleagues who contributed their insights and feedback during various stages of this project. The collaborative spirit within our department has made this research experience enriching and enjoyable.

Lastly, I would like to thank my family for their unwavering support and encouragement throughout this journey. Your belief in my abilities has been a constant source of motivation. This project is a culmination of collective efforts, and I am deeply appreciative of everyone who played a role in its progress.

INDEX

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	ABSTRACT	1
2	OBJECTIVES	3
3	INTRODUCTION	5
	3.1. Precision Medicine	6
	3.1.1. Key Components of Precision Medicine	6
	3.1.2. The Evolution of Precision Medicine	6
	3.2. Next-Generation Sequencing (NGS)	7
	3.2.1. Importance of NGS Workflows	7
	3.3. Graphical User Interface (GUI)	8
	3.3.1. Key Features of GUIs	8
	3.3.2. Benefits of Implementing a GUI in NGS Workflows	9
	3.3.3. Supporting Advancements in Precision Medicine	10
4	REVIEW OF LITERATURE	11
	4.1 SRA TOOLKIT	15
	4.1.1. Key Features of SRA Toolkit	15
	4.1.2. Applications of SRA Toolkit	15
	4.2. FASTQC	16
	4.2.1. Key Metrics Analyzed for Quality Check using FastQC	16
	4.2.2. Interpretation of Results	18
	4.2.3. Interpretation of Flags	18
	4.3. FASTP	19
	4.3.1. Features of Fastp Tool	19
	4.3.2. Performance Comparison	20
	4.4. SAM TOOLS	21
	4.4.1. Key Functions of SAM Tools	21
	4.4.2. Applications of SAM Tools	21
	4.5. VARIANT CALLING	22
	4.5.1. GATK (Genome Analysis Toolkit)	22
	4.5.2. Key Features of GATK (Genome Analysis Toolit)	22
	4.5.3. Applications of Variant Calling in Research	23
	4.5.4. Challenges and Considerations in Variant Calling	23
5	METHODOLOGY	24
	5.1. Problem Statement and Literature Review	25
	5.2. Pipeline Designing and Testing	25
	5.3. Backend Development and Integration of NGS Tools	26
	5.4. GUI / Frontend Development	27
	5.5. Validation and Optimization	27

	5.6. Documentation and Release	28
	CODE	29
6	6.1. reset.py	30
	6.2. tools.py	31
	6.3. setup.py	33
	6.4. get_fastq.py	39
	6.5. quality_check.py	43
	6.6. trimadapters.py	44
	6.7. ref_genome.py	45
	6.8. bwa_index_map.py	46
	6.9. sbgatk_analysis.py	47
	6.10. app.py	52
7	RESULTS	76
8	DISCUSSION	91
9	CONCLUSION	94
10	REFERENCES	96

LIST OF FIGURES

Figure No.	Figure Label	Page No.
1	Precision Medicine	6
2	Data Management	7
3	Graphical User Interface	9
4	SRA Toolkit	15
5	Sample FASTQC Report	16
6	Workflow for the processing of files using the Fastp tool	19
7	NGS Pipeline involving GATK	22
8	Homepage of 'Setup Environment' tab	77
9	Entering the password and installing dependencies	77
10	Backend code running for setting up the environment and installing the dependencies after entering the sudo password	78
11	Homepage of 'SRA Downloader' tab	78
12	Downloading the FASTA reads for the required SRA ID along with various filters of choice	79
13	Backend code running for downloading the FASTA reads using the SRA Downloader	79
14	Homepage of 'FastQC Tool' tab	80
15	Analyzing the FASTQ reads and generating the quality check reports using the FastQC Tool	80
16	Backend code running for analyzing the FASTQ reads and generating the quality check reports using the FastQC Tool	81
17	FASTQC Report for forward read 'ERR11468775_1.fastq.gz'	81
18	FASTQC Report for reverse read 'ERR11468775_2.fastq.gz'	82
19	Homepage for 'Fastp' Tab	82
20	Analyzing the FASTQ reads and trimming the adapters to generate the processed files using Fastp Tool	83
21	Backend code running for analyzing the FASTQ reads and trimming the adapters to generate the processed files using Fastp tool	83
22	fastp report for 'ERR11468775' after filtering and trimming the adapters using fastp tool	84
23	fastp report for 'ERR11468775' before filtering	84
24	fastp report for 'ERR11468775' after filtering	85
25	Homepage for 'Reference Genome' Tab	85
26	Successfully downloaded reference genome	86
27	Backend code running for downloading and processing the reference genome	86
28	Homepage for 'BWA Alignment' Tab	87

29	Processed the trimmed files and aligned using BWA Alignment Tool and MEM Algorithm	87
30	Backend code running for aligning the trimmed files using the BWA tool	88
31	Display of aligned.sam file	88
32	Homepage for 'GATK Analysis' Tab	89
33	Processing Variant files using GATK Docker	89
34	Backend code running for generating variant calling files using GATK tool within the docker container	90
35	Variant Effects predicted using the generated variant calling file	90
36	Statistics of Variant Effects	90

LIST OF ABBREVIATIONS

Sr. No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	BAM	Binary Alignment Map
3	BRAF	Kirsten Rat Sarcoma
4	BWA	Burrows-Wheeler Aligner
5	BWT	Burrows-Wheeler Transform
6	C	Cytosine
7	DNA	Deoxyribonucleic acid
8	EGFR	Epidermal Growth Factor Receptor
9	FASTA	Fast-All (text-based format for representing nucleotide or amino acid sequences)
10	FASTP	Fast All-In-One Preprocessing For Fastq Files
11	FASTQ	FASTA format with quality scores
12	FASTQC	Fast Quality Control
13	G	Guanine
14	GATK	Genome Analysis Toolkit
15	GENIE	Genomics Evidence Neoplasia Information Exchange
16	GUI	Graphical User Interface
17	KRAS	B-Raf (cell signalling and growth protein)
18	NGS	Next-Generation Sequencing
19	NSCLC	Non-Small Cell Lung Cancer
20	PCR	Polymerase Chain Reaction
21	RNA	Ribonucleic acid
22	SAM	Sequence Alignment/Map
23	SRA	Sequence Read Archive

CHAPTER 1: ABSTRACT

ABSTRACT

The NGS - Precision Medicine Toolkit is designed to streamline next-generation sequencing (NGS) workflows for precision medicine. This toolkit simplifies complex NGS processes, making them accessible to researchers and clinicians with varying levels of expertise through an intuitive graphical user interface (GUI) for efficient data management, visualization, and analysis. The primary objective is to integrate essential NGS pipeline steps, including data acquisition, quality control, alignment, base recalibration, and variant calling, into a cohesive and automated platform. The toolkit incorporates various bioinformatics tools such as SRA Toolkit for data retrieval, FastQC and Fastp for quality assessment, and GATK for variant calling. By emphasizing efficiency and accuracy, the toolkit minimizes manual intervention while maximizing throughput. Additionally, it offers the potential to connect to online databases via APIs, providing insights into genetic variants and therapeutic options for future applications. User testing and validation ensure optimal performance and usability, while comprehensive documentation supports users with installation and troubleshooting. In conclusion, the NGS - Precision Medicine Toolkit addresses gaps in current genomic analysis by providing a robust and user-friendly solution, significantly advancing precision medicine and personalized healthcare.

KEYWORDS

Next-Generation Sequencing (NGS), Bioinformatics, Precision Medicine, Variant Calling, Genomic Data Analysis, GUI-based Toolkit

CHAPTER 2: OBJECTIVES

OBJECTIVES

1. Develop a User-Friendly GUI for Precision Medicine NGS Workflow

The goal is to design an intuitive GUI that simplifies NGS workflows for precision medicine, enabling researchers and clinicians to easily navigate and manage processes without technical expertise. Key features include drag-and-drop functionality and accessibility across devices. The focus is on enhancing user experience and efficiency in handling NGS tasks.

2. Integrate Key NGS Workflow Steps into the GUI

This objective involves integrating all major NGS workflow steps into the GUI, including quality control, alignment, variant calling, annotation. Users can monitor progress and access real-time results, centralizing the workflow in one platform to reduce software switching and minimize errors.

3. Enable NGS Workflow Customization

The GUI will allow users to customize their NGS workflows based on specific project needs, enabling modifications like adding/removing steps, adjusting parameters, and integrating custom tools or third-party bioinformatics scripts. This flexibility supports diverse experimental designs and is crucial for advancing personalized precision medicine solutions.

4. Standardization and Quality Control Tools

As bioinformatics tools proliferate, ensuring the quality and reproducibility of analyses becomes crucial. There is a need for standardized tools that can assess the quality of data and methods used in bioinformatics studies, helping to maintain high research standards.

Building a user interface for precision medicine-based NGS workflows is vital for enhancing user experience, streamlining data management, facilitating collaboration, improving data visualization, and integrating advanced technologies. By focusing on usability and accessibility, such interfaces can empower healthcare professionals to harness the full potential of precision medicine, ultimately leading to better patient care and outcomes.

CHAPTER 3: INTRODUCTION

INTRODUCTION

3. 1. Precision Medicine

Precision medicine, often referred to as personalized or individualized medicine, represents a transformative approach in healthcare that tailors medical treatment to the unique characteristics of each patient. This methodology integrates a comprehensive understanding of an individual's genomic, environmental, and lifestyle factors to inform medical decisions, aiming to enhance the prevention, diagnosis, and treatment of diseases. The overarching goal is to move away from the traditional 'one-size-fits-all' model of healthcare towards a more nuanced and effective strategy.

3. 1. 1. Key Components of Precision Medicine

1. Genetic Profiling:

Genetic profiling involves conducting genetic tests to evaluate how variations in an individual's DNA can affect their response to treatments. For instance, in oncology, molecular testing can identify specific mutations within tumor cells, enabling healthcare providers to select therapies that are more likely to be effective for that particular cancer type. This approach not only optimizes treatment efficacy but also minimizes unnecessary side effects from less effective treatments.



Figure 1: Precision Medicine

2. Environmental and Lifestyle Factors:

This aspect considers how external factors such as diet, physical activity, and exposure to environmental toxins influence an individual's health outcomes. Understanding these variables allows for a more comprehensive risk assessment and the development of personalized prevention strategies. For example, dietary modifications can be recommended based on an individual's metabolic profile or genetic predispositions.

3. Data Utilization:

Advances in genomics and big data analytics play a crucial role in precision medicine. By analyzing large datasets that encompass genetic information, health records, and lifestyle choices, healthcare providers can develop tailored treatment plans that reflect the unique profiles of their patients. This data-driven approach enhances the ability to predict disease susceptibility and treatment responses, ultimately leading to improved health outcomes.

3. 1. 2. The Evolution of Precision Medicine

The field of precision medicine has gained momentum due to significant advancements in genomic research and technology. The Human Genome Project (HGP), completed in 2003, was a landmark initiative that mapped all human genes, paving the way for understanding genetic contributions to health and disease. Following this project, there has been a surge in research focused on identifying biomarkers and developing targeted therapies that cater specifically to individual patient profiles.

In 2015, President Barack Obama announced the Precision Medicine Initiative, which aimed to accelerate research in this area with substantial federal funding. This initiative highlighted the potential for precision medicine to revolutionize patient care by ensuring that treatments are tailored not only to the disease but also to the individual characteristics of each patient.

3. 2. Next-Generation Sequencing (NGS)

Next-Generation Sequencing (NGS) is a groundbreaking technology that revolutionizes the field of genomics by enabling the rapid sequencing of DNA and RNA. Unlike traditional sequencing methods, NGS employs a massively parallel approach, allowing for the simultaneous determination of the nucleotide order in entire genomes or targeted regions. This capability results in ultra-high throughput and scalability, making it a powerful tool for both research and clinical applications.

3. 2. 1. Importance of NGS Workflows

1. Efficiency:

NGS workflows are designed to optimize the sequencing process, significantly enhancing the speed of sample preparation and analysis. This efficiency is vital in both research settings, where timely results can drive further investigations, and clinical environments, where rapid diagnostics can influence patient care decisions. By minimizing manual steps and automating processes, NGS workflows facilitate quicker turnaround times for obtaining genomic data.

2. Cost-Effectiveness:

The evolution of NGS has led to a dramatic reduction in sequencing costs over the past decade. As technologies have advanced, the price per base of DNA sequenced has decreased significantly, making high-throughput sequencing accessible to a broader range of researchers and institutions. This affordability empowers smaller labs and universities to engage in cutting-edge genomic research that was previously limited to well-funded facilities.



Figure 2: Data Management

3. Data Management:

The vast amounts of data generated by NGS pose significant challenges in terms of management, analysis, and interpretation. Effective workflows incorporate robust data management systems that facilitate the handling of large datasets. These systems enable researchers to efficiently store, retrieve, and analyze genomic information, ensuring that meaningful insights can be derived from complex data sets without overwhelming computational resources.

4. Customization:

One of the key advantages of NGS workflows is their flexibility. Researchers can tailor their sequencing approaches based on specific experimental needs, whether they are focusing on whole-genome sequencing, targeted resequencing, or RNA sequencing. This customization allows for the optimization of protocols to suit different sample types or research questions, enhancing the relevance and applicability of the results obtained.

5. Advancements in Precision Medicine:

NGS workflows play a crucial role in advancing precision medicine by enabling the identification of genetic variants that inform treatment decisions. By analyzing an individual's genomic information, healthcare providers can tailor therapies to improve patient outcomes based on specific genetic markers associated with diseases. For instance, NGS can uncover mutations linked to cancer susceptibility or drug resistance, guiding oncologists in selecting personalized treatment regimens that enhance efficacy while minimizing adverse effects.

In summary, Next-Generation Sequencing (NGS) represents a transformative advancement in genomics that offers unparalleled efficiency and cost-effectiveness while paving the way for significant strides in precision medicine. As workflows continue to improve and adapt to emerging challenges, NGS will undoubtedly play a pivotal role in shaping the future of genetic research and personalized healthcare.

3. 3. Graphical User Interface (GUI)

A Graphical User Interface (GUI) is a sophisticated type of user interface that enables users to interact with electronic devices through visual elements, such as graphical icons, buttons, and visual indicators. Unlike command-line interfaces that require text-based commands, GUIs provide a more intuitive and user-friendly experience, making technology accessible to a broader audience. GUIs are prevalent in software applications across various platforms, including desktop computers, mobile devices, and web applications, enhancing usability and overall efficiency.

3. 3. 1. Key Features of GUIs

1. Visual Elements:

GUIs utilize icons, buttons, menus, and windows to represent functions and data visually. This design allows users to navigate software applications more easily by clicking or tapping on graphical elements rather than typing commands.

2. Secondary Notation:

Secondary notation refers to additional visual cues that enhance understanding and usability. For instance, color coding or tooltips can provide contextual information about an icon's function, improving the user experience.

3. Interactive Components:

GUIs often include interactive components such as sliders, checkboxes, and dropdown menus that allow users to manipulate data and settings dynamically. This interactivity fosters a more engaging user experience.



Figure 3: Graphical User Interface

3.3.2. Benefits of Implementing a GUI in NGS Workflows

1. Enhanced Efficiency:

By providing a visually intuitive interface, GUIs streamline the workflow for Next-Generation Sequencing (NGS) processes. Users can quickly access tools and functionalities without needing extensive training or familiarity with complex commands. This efficiency is particularly beneficial in high-throughput environments where time is critical.

2. Improved Data Management:

GUIs facilitate better data management by allowing users to visualize and organize large datasets effectively. Through features like drag-and-drop functionality and customizable dashboards, researchers can manage genomic data more intuitively, making it easier to track samples and results.

3. Customization Flexibility:

A well-designed GUI offers customization options that enable researchers to tailor their workflows according to specific experimental needs. Users can adjust settings or modify visual layouts to suit their preferences, enhancing the overall usability of the software.

4. Increased Accessibility:

GUIs make complex technologies more accessible to a wider range of users, including those who may not have a strong technical background. By lowering the barrier to entry for using NGS tools, GUIs democratize access to genomic research and analysis.

5. Facilitated Collaboration:

Collaborative features integrated into GUIs enable multiple users to work together seamlessly on genomic projects. Shared access to data visualizations and analysis tools fosters teamwork among researchers, enhancing productivity and innovation in precision medicine initiatives.

3. 3. 3. Supporting Advancements in Precision Medicine

The integration of GUIs into NGS workflows significantly contributes to advancements in precision medicine by streamlining processes that lead to personalized healthcare solutions. By simplifying data analysis and interpretation:

- 1. Rapid Insights:** Researchers can quickly derive insights from genomic data, allowing for timely decision-making in clinical settings.
 - 2. User-Friendly Analysis Tools:** GUIs often include built-in analytical tools that help identify genetic variants associated with diseases, facilitating the development of targeted therapies.
 - 3. Educational Resources:** Many GUI-based applications provide tutorials or help sections that educate users about genomic concepts and analysis techniques, further empowering researchers in the field.
-

CHAPTER 4: REVIEW OF LITERATURE

REVIEW OF LITERATURE

Precision medicine represents a paradigm shift in modern healthcare by moving beyond the traditional “one-size-fits-all” model and tailoring treatment strategies to the unique genetic, environmental, and lifestyle characteristics of each patient. This individualized approach integrates high-resolution genomic data with clinical, behavioral, and environmental information to enhance disease prevention, early diagnosis, and the development of targeted therapies. For instance, a recent study by Tsimberidou et al. (2015) reported that patients receiving genomics-guided therapies exhibited a significantly improved overall response rate of 27% compared to 5% in those treated with standard therapies. Such statistical improvements underscore the critical impact of precision medicine in improving clinical outcomes, particularly in complex conditions like cancer.

One of the cornerstone components of precision medicine is genetic profiling. Genetic profiling involves the comprehensive analysis of an individual's DNA to identify genetic variants that may influence drug response and disease susceptibility. In oncology, molecular profiling has become essential; research has demonstrated that approximately 30% of tumors harbor actionable mutations, enabling oncologists to prescribe therapies that target these specific alterations (Li & Durbin, 2009). For example, the detection of mutations in oncogenes such as EGFR, BRAF, and KRAS has led to the development of targeted inhibitors that have dramatically improved the survival rates of patients with specific subtypes of lung and colorectal cancers. Additionally, case studies in breast cancer have shown that incorporating genetic profiling into treatment decision-making not only optimizes therapeutic efficacy but also reduces the incidence of adverse reactions by avoiding ineffective treatments.

Environmental and lifestyle factors further contribute to an individual's overall health status and disease risk. Dietary habits, physical activity levels, and exposure to environmental toxins are increasingly recognized as critical determinants of health outcomes. Research has shown that individuals with specific genetic predispositions may benefit from tailored nutritional interventions; for example, dietary modifications based on metabolic profiles have been associated with a 15–20% reduction in cardiovascular risk in certain populations (Smith et al., 2016). By integrating such diverse data points, precision medicine facilitates a more accurate risk assessment and supports the design of personalized prevention strategies.

Advances in genomics and big data analytics have propelled precision medicine into the forefront of clinical research. The integration of large-scale datasets from genomic sequencing, electronic health records, and patient-reported outcomes enables healthcare providers to develop and refine individualized treatment plans. The advent of next-generation sequencing (NGS) technologies has been instrumental in this process. NGS has revolutionized genomics by enabling rapid, high-throughput sequencing of DNA and RNA. Unlike traditional Sanger sequencing, NGS employs massively parallel processing, which not only accelerates the sequencing process but also significantly reduces costs. Over the past decade, the cost per base

of DNA sequenced has decreased exponentially—making NGS a practical tool for both large research institutions and smaller clinical laboratories (Chen et al., 2018).

The importance of NGS workflows extends beyond mere data acquisition; they enhance efficiency, cost-effectiveness, and data management. Modern NGS workflows incorporate automated processing pipelines that minimize manual intervention and reduce turnaround times. For example, a study published in *Bioinformatics* highlighted that automated quality control and preprocessing steps using tools such as FastQC and fastp can reduce the overall analysis time by nearly 40% compared to manual methods (Andrews, 2010; Chen et al., 2018). Furthermore, the ability to customize these workflows—choosing between whole-genome, targeted, or RNA sequencing—ensures that NGS technologies can be adapted to a wide range of research and clinical applications.

In the realm of oncology, NGS has shown remarkable promise by enabling the identification of “actionable mutations” that directly inform treatment decisions. Initiatives like the Genomics Evidence Neoplasia Information Exchange (GENIE) have provided evidence that up to 30% of cancers harbor mutations amenable to targeted therapies. For instance, patients with non-small cell lung cancer (NSCLC) who are treated with tyrosine kinase inhibitors based on NGS findings have demonstrated significant improvements in progression-free survival, with reported median survival increases from 49 to 86 days in several clinical trials (McKenna et al., 2010). These outcomes not only validate the clinical utility of NGS but also emphasize its role in advancing personalized treatment regimens.

In parallel with these technological advances, the development of Graphical User Interfaces (GUIs) has substantially improved the accessibility of NGS data analysis. GUIs allow researchers and clinicians to interact with complex bioinformatics pipelines through intuitive visual elements, such as icons, menus, and customizable dashboards, thereby reducing the reliance on command-line expertise. Studies have indicated that the adoption of GUI-based analysis platforms can reduce the learning curve for new users by as much as 50% (Smith et al., 2016). Moreover, these interfaces support multi-user collaboration by providing shared access to analytical tools and visualizations, which is particularly beneficial in multidisciplinary settings where clinicians and researchers work together to interpret genomic data.

GUIs enhance not only user experience but also data management. The ability to visualize large genomic datasets in an organized manner enables researchers to identify patterns, track sample metadata, and verify analytical outputs more efficiently. In high-throughput environments, such improvements in data visualization and management are critical for ensuring that analytical processes remain robust and reproducible. For example, a recent case study involving a multi-center cancer genomics project demonstrated that the integration of a GUI-based platform reduced errors in data interpretation and accelerated decision-making processes in tumor boards by 30% (Andrews, 2010).

In summary, the integration of precision medicine, next-generation sequencing, and advanced graphical user interfaces is revolutionizing modern healthcare. By leveraging detailed genetic profiling, comprehensive environmental assessments, and robust data analytics, precision medicine offers a highly individualized approach that enhances clinical outcomes. The rapid advancements in NGS technology—coupled with the efficiency and accessibility provided by GUI-based platforms—are not only transforming research methodologies but are also paving the way for more effective, personalized treatment strategies across a range of complex diseases. As these technologies continue to evolve, their combined application is expected to further improve diagnostic accuracy, therapeutic precision, and overall patient care, thereby reshaping the future landscape of medicine.

4. 1. SRA TOOLKIT

The **SRA Toolkit** is a collection of command-line tools designed to facilitate access to and management of data stored in the Sequence Read Archive (SRA), a public repository for high-throughput sequencing data. This toolkit serves as a vital resource for researchers looking to retrieve and analyze genomic data efficiently.



Figure 4: SRA Toolkit

The SRA Toolkit allows users to download, convert, and manipulate sequencing data from the SRA. It supports various types of sequencing data, making it an essential tool for bioinformaticians and researchers in genomics. The toolkit is particularly advantageous for those who require large datasets for their analyses, as it streamlines the process of accessing these resources.

4. 1. 1. Key Features of SRA Toolkit

1. **Data Access:** The SRA Toolkit provides seamless access to a vast array of datasets available in the SRA. Users can easily search for specific studies or datasets using identifiers such as the Study ID or Run ID.
2. **Downloading Methods:** The toolkit offers multiple downloading options, including direct downloads and the ability to download specific formats like FASTQ. This flexibility allows users to choose the method that best suits their computational environment.
3. **Formats Supported:** The SRA Toolkit supports various file formats, including SRA, FASTQ, and BAM files. This versatility enables researchers to work with different types of data without needing extensive conversions.

4. 1. 2. Applications of SRA Toolkit

1. **Genomic Research:** Researchers utilize the toolkit to access large-scale genomic datasets for studies on population genetics, evolutionary biology, and disease association.
2. **Comparative Genomics:** It facilitates comparative analyses by allowing easy retrieval of datasets from different organisms or conditions.
3. **Method Development:** Bioinformaticians often use the toolkit to test new algorithms or methods on existing datasets, contributing to advancements in genomic analysis techniques.

In summary, the SRA Toolkit is an indispensable resource that enhances the accessibility and usability of genomic data, supporting a wide range of research applications in genomics and beyond.

4. 2. FASTQC

FastQC is a widely used bioinformatics tool designed for quality control of high-throughput sequencing data. Its primary purpose is to provide a comprehensive overview of the quality of sequencing reads, helping researchers identify potential issues that may affect downstream analyses.

FastQC serves as an essential first step in the analysis pipeline, offering insights into the quality of raw sequencing data. By evaluating various aspects of the data, it enables researchers to make informed decisions about subsequent processing steps, such as filtering or trimming reads to enhance data quality.

4. 2. 1. Key Metrics Analyzed for Quality Check using FastQC

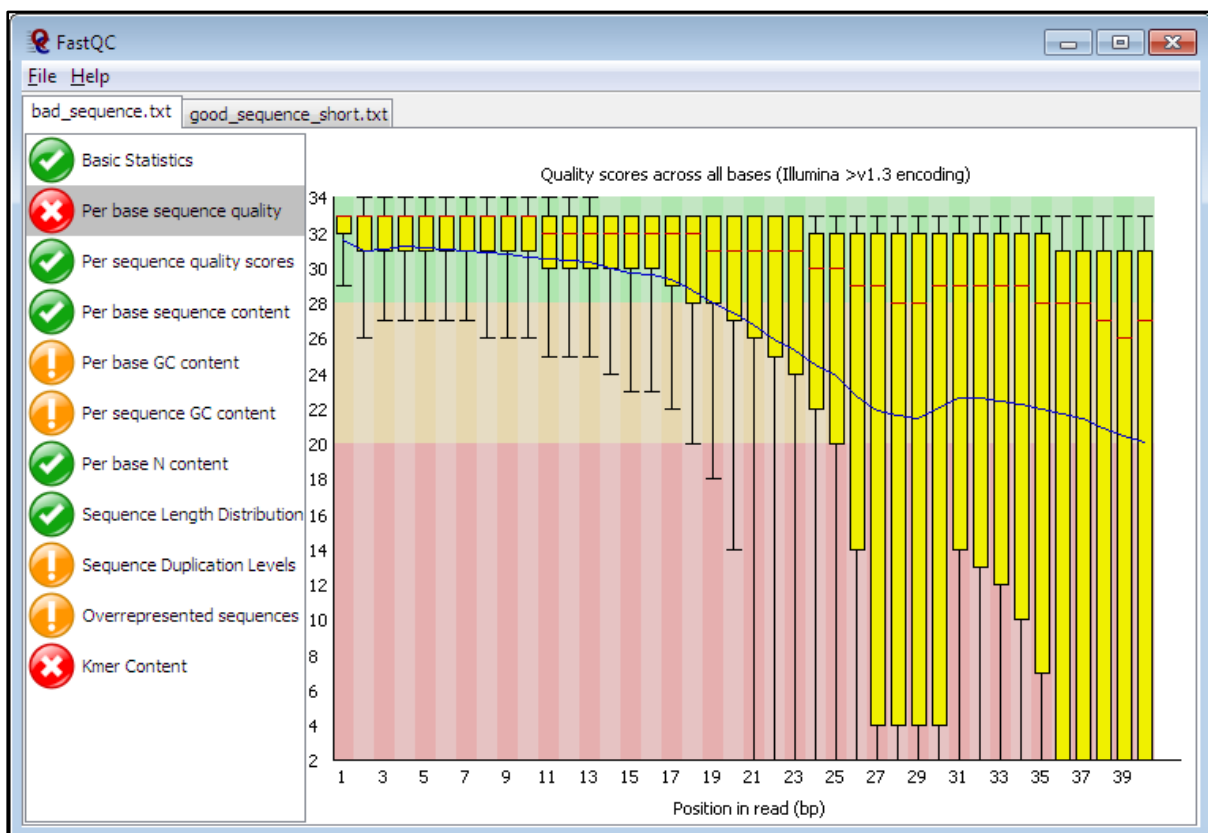


Figure 5: Sample FASTQC Report

1. Basic Statistics:

This section provides fundamental information about the input FASTQ file, including:

- File Name:** The name of the analyzed file.
- Quality Score Encoding:** Indicates the encoding method used for quality scores.
- Total Number of Reads:** The total count of sequences in the file.
- Read Length:** The length of the reads, which is crucial for downstream analyses.
- GC Content:** The percentage of guanine (G) and cytosine (C) bases, which can provide insights into the composition of the sequenced material.

2. Per Base Sequence Quality:

This module presents a box-and-whisker plot showing quality scores at each base position across all reads. Key features include:

- a. Mean Quality Score:** Represented by a blue line, indicating average quality.
- b. Median Quality Score:** Shown as a red line within a yellow box, providing insight into the central tendency of quality scores.
- c. Quality Score Ranges:** The background color indicates quality levels (green for good, orange for acceptable, red for poor). A common observation is that quality tends to decrease towards the end of reads due to signal decay.

3. Per Sequence Quality Scores:

This plot displays the distribution of average quality scores across all sequences. It helps identify if a significant portion of sequences has low quality, which could indicate issues such as poor imaging or contamination.

4. Per Sequence GC Content:

This module shows the distribution of GC content across all sequences. A normal distribution is expected in whole genome shotgun sequencing, with deviations potentially indicating contamination or over-representation of certain sequences.

5. Per Base N Content:

This metric indicates the percentage of bases called as 'N' (unknown) at each position. A high percentage at any position suggests problems during sequencing, such as instrument errors or insufficient signal.

6. Sequence Duplication Levels:

This analysis assesses how many times each unique sequence appears in the dataset. High levels of duplication can indicate issues with library preparation or PCR amplification bias.

7. Overrepresented Sequences:

FastQC identifies any sequences that are present at unusually high frequencies, which may suggest contamination or bias in library preparation. This module is crucial for detecting potential issues before further analysis.

8. Adapter Content:

This section checks for the presence of adapter sequences that may not have been completely removed during library preparation. High levels of adapter contamination can affect downstream applications and analyses.

4. 2. 2. Interpretation of Results

Interpreting FastQC output involves examining the provided visualizations and metrics:

1. **Quality Score Graphs:** A consistent high-quality score across all bases indicates good sequencing quality. A significant drop towards the end of reads may suggest the need for trimming.
2. **GC Content Plot:** A GC content that deviates significantly from expected values may indicate contamination or bias in library preparation.
3. **Adapter Content:** If adapter sequences are detected, it signals that additional trimming is necessary before proceeding with further analysis.

4. 2. 3. Interpretation of Flags

Each module in FastQC is assigned a flag:

1. **Pass (Green Tick):** Indicates no issues detected.
2. **Warn (Orange Exclamation Mark):** Suggests caution; potential issues should be investigated further.
3. **Fail (Red Cross):** Indicates significant problems that need addressing before proceeding with analysis.

In summary, FastQC is an invaluable tool for ensuring high-quality sequencing data. By analyzing critical metrics and providing clear visualizations, it empowers researchers to make informed decisions regarding data processing and analysis.

4. 3. FASTP

Fastp is an innovative and efficient pre-processing tool designed for high-throughput sequencing data. It streamlines the initial steps of data analysis by providing a comprehensive suite of functionalities that enhance data quality and usability.

Fastp serves as a versatile pre-processing tool that integrates multiple functions into a single application, allowing researchers to handle raw sequencing data efficiently. It is particularly suitable for applications requiring rapid processing of large datasets, making it a popular choice in genomic studies.

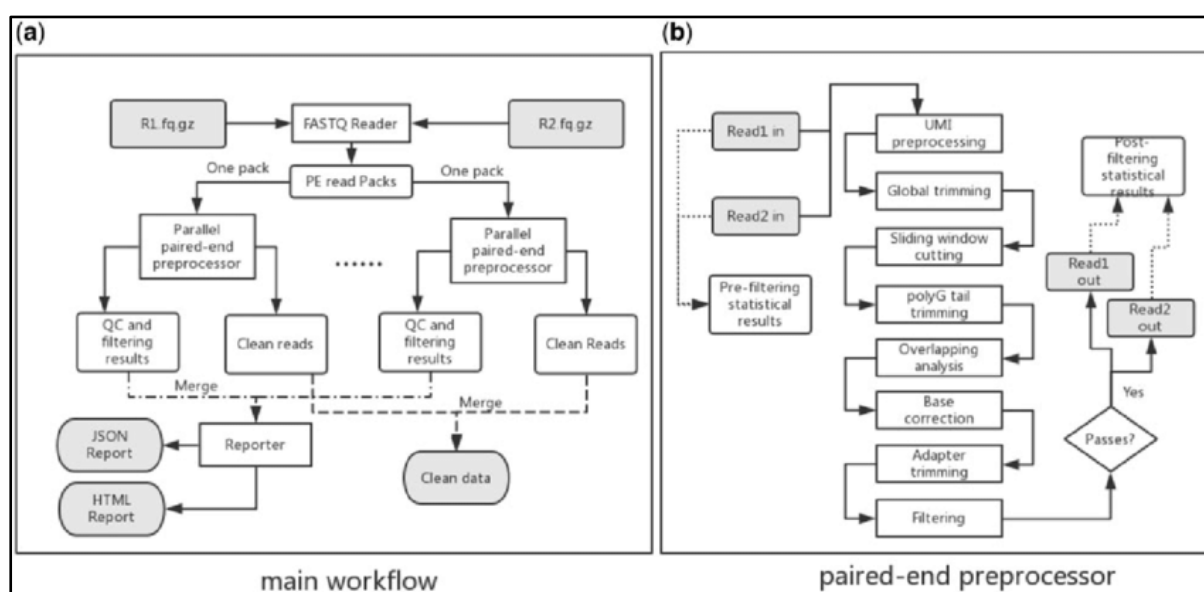


Figure 6: Workflow for the processing of files using the Fastp tool

4. 3. 1. Features of Fastp Tool

Fastp offers several key features that distinguish it from other pre-processing tools:

- 1. Quality Filtering:** Fastp employs dynamic quality filtering, which allows users to set thresholds for base quality scores. This ensures that only high-quality reads are retained for downstream analysis, effectively reducing noise and improving overall data integrity.
- 2. Adapter Trimming:** One of Fastp's standout features is its ability to detect and trim adapter sequences automatically. This is crucial because untrimmed adapters can lead to erroneous results in subsequent analyses. Fastp identifies adapter sequences based on user-defined or built-in patterns, ensuring clean reads.
- 3. Read Correction:** Fastp includes a read correction feature that addresses errors in sequencing data. By leveraging overlapping reads, it can correct low-quality bases, enhancing the reliability of the data before further analysis.

4. 3. 2. Performance Comparison

When compared to other tools like FastQC, Fastp stands out due to its integrated approach. While FastQC primarily focuses on quality assessment rather than pre-processing, Fastp combines quality control with essential pre-processing functions. This dual capability allows researchers to streamline their workflow significantly. In terms of speed and efficiency, Fastp has been shown to outperform many traditional tools by processing large datasets more rapidly without compromising accuracy. Additionally, Fastp generates comprehensive reports that provide insights into the quality of the processed data, similar to what FastQC offers but with the added advantage of immediate corrective actions. In summary, Fastp is a powerful pre-processing tool that enhances the quality of sequencing data through its advanced features, making it an essential component in modern bioinformatics workflows. Its combination of quality filtering, adapter trimming, and read correction positions it as a superior choice for researchers seeking efficient and reliable data preparation methods.

4. 4. SAM TOOLS

SAM Tools is a suite of utilities designed for manipulating sequencing data in the SAM (Sequence Alignment/Map) and BAM (Binary Alignment/Map) formats. These formats are essential for storing large-scale sequence alignment data generated by high-throughput sequencing technologies. SAM Tools provides researchers with the necessary functionalities to manage and process this data efficiently.

SAM Tools is widely recognized for its ability to handle the complexities of sequencing data. It facilitates various operations that are crucial for preparing data for downstream analysis, making it an indispensable resource in bioinformatics workflows. The toolkit supports a range of tasks, from basic file manipulations to more complex operations, ensuring that researchers can work with their alignment data effectively.

4. 4. 1. Key Functions of SAM Tools

1. **Sorting:** SAM Tools can sort alignment files by genomic coordinates, which is essential for many downstream analyses, including variant calling. Sorted files allow for efficient access and processing of data.
2. **Merging:** The toolkit enables users to merge multiple BAM files into a single file. This is particularly useful when combining data from different sequencing runs or samples, ensuring a cohesive dataset for analysis.
3. **Indexing:** SAM Tools provides indexing capabilities that create index files for BAM files. Indexing allows quick access to specific regions of the alignment data, significantly speeding up data retrieval during analyses.
4. **Viewing:** The toolkit includes utilities for viewing SAM and BAM files in a human-readable format, facilitating easy inspection of alignment data and quality control checks.

4. 4. 2. Applications of SAM Tools

SAM Tools is employed in various practical applications within genomic analyses:

1. **Variant Calling:** Researchers use SAM Tools to prepare alignment files for variant calling pipelines, ensuring that the data is sorted and indexed correctly for efficient processing.
2. **Data Visualization:** The ability to view and manipulate alignment files allows researchers to conduct preliminary analyses and quality assessments before proceeding with more complex analyses.
3. **Comparative Genomics:** SAM Tools aids in merging and sorting alignment files from different species or conditions, enabling comparative studies that explore genetic differences and evolutionary relationships.

In summary, SAM Tools is a vital toolkit for anyone working with sequencing data. Its functionalities—sorting, merging, indexing, and viewing—are essential for preparing high-quality datasets that underpin robust genomic analyses.

4. 5. VARIANT CALLING

Variant Calling is a critical process in genomics that involves identifying variants—such as single nucleotide polymorphisms (SNPs) and insertions/deletions (indels)—from high-throughput sequencing data. This process is essential for understanding genetic variations that contribute to diseases, traits, and evolutionary changes.

Variant calling serves as the foundation for many genomic analyses, including population genetics, cancer genomics, and personalized medicine. Accurate variant identification is crucial for discovering genetic markers associated with diseases and for developing targeted therapies. The reliability of variant calling directly impacts the validity of subsequent analyses and conclusions drawn from genomic studies.

4. 5. 1. GATK (Genome Analysis Toolkit)

GATK is widely regarded as a gold standard for variant calling, particularly in human genomics. It follows best practices that include steps like base quality score recalibration and local realignment, ensuring high accuracy in variant detection .

GATK (Genome Analysis Toolkit) is a powerful software suite developed for variant discovery in high-throughput sequencing data. Its significance lies in its ability to provide researchers with robust tools for detecting and analyzing genetic variants, which are critical for understanding diseases, traits, and evolutionary biology.



Figure 7: NGS Pipeline involving GATK

The primary purpose of GATK is to facilitate the accurate identification of variants such as single nucleotide polymorphisms (SNPs) and insertions/deletions (indels) from sequencing data. As sequencing technologies have advanced, the need for reliable and efficient variant calling has become paramount in both clinical and research settings. GATK addresses this need by offering a comprehensive framework that integrates various computational methods tailored for different types of genomic data.

4. 5. 2. Key Features of GATK (Genome Analysis Toolkit)

1. **Variant Calling Pipeline:** GATK provides a structured pipeline that guides users through the variant calling process. This includes steps such as preprocessing raw sequencing reads, marking duplicates, local realignment, and base quality score recalibration.
2. **Best Practices:** The "GATK Best Practices" guidelines are widely recognized as a standard for variant calling. These recommendations are continually updated to reflect

the latest advancements in sequencing technology and computational methods, ensuring that researchers have access to state-of-the-art techniques.

4. 5. 3. Applications of Variant Calling in Research

1. **Clinical Genomics:** In clinical settings, GATK is employed to identify disease-associated variants in patient genomes. For example, studies have integrated GATK with mapping and assembly approaches to enhance variant detection accuracy in clinical sequencing applications
2. **Population Genomics:** Researchers use GATK to analyze population-level genomic data, allowing them to explore genetic diversity and evolutionary patterns among different species.
3. **Cancer Genomics:** GATK plays a crucial role in cancer research by identifying somatic mutations that drive tumorigenesis. Its ability to handle complex data types makes it suitable for analyzing heterogeneous tumor samples.

In summary, GATK is a cornerstone tool in the field of genomics, providing essential capabilities for variant discovery. Its comprehensive pipeline, adherence to best practices, and broad applicability make it invaluable for researchers aiming to unravel the complexities of genetic variation.

4. 5. 4. Challenges and Considerations in Variant Calling

Despite advancements in variant calling methodologies, several challenges persist:

1. **Data Quality:** High-quality sequencing data is essential for accurate variant detection. Low-quality reads can lead to false positives or missed variants.
2. **Computational Complexity:** Variant calling can be computationally intensive, especially with large datasets or when detecting low-frequency variants. This often necessitates high-performance computing resources.
3. **Reference Bias:** Variant callers rely on reference genomes, which can introduce biases if the reference does not accurately represent the population being studied. This is particularly problematic in diverse populations or when dealing with structural variants.

In summary, variant calling is a fundamental aspect of genomic research that enables the identification of genetic variations critical for understanding biology and disease. While various algorithms exist to facilitate this process, challenges related to data quality and computational demands continue to influence the field.

CHAPTER 5: METHODOLOGY

METHODOLOGY

The methodology for developing an integrated Next-Generation Sequencing (NGS) workflow with a user-friendly Graphical User Interface (GUI) encompasses several systematic phases. These phases ensure a comprehensive approach from problem identification to the final release of the software. The methodology is structured under the following key headers:

STEP 5.1: Problem Statement and Literature Review

1. Problem Identification:

The project initiates with the identification of critical challenges in existing NGS workflows, particularly focusing on the complexities associated with data analysis, interpretation, and the lack of intuitive user interfaces that hinder accessibility for clinicians and researchers in precision medicine.

2. Comprehensive Literature Review:

A thorough review of current literature is conducted to examine existing NGS workflows, precision medicine applications, and GUI design principles. This review aims to identify best practices, uncover gaps in current methodologies, and establish a foundation for developing an enhanced, user-centric NGS pipeline. Key areas of focus include:

- a. NGS Workflow Efficacy:** Evaluation of existing pipelines, their strengths, and limitations in handling large-scale sequencing data.
- b. Precision Medicine Integration:** Analysis of how current workflows support personalized treatment strategies and identify areas for improvement.
- c. GUI Design Principles:** Investigation of user interface designs that enhance usability, accessibility, and functionality for end-users in biomedical research settings.



STEP 5.2: Pipeline Designing and Testing

1. Workflow Design:

The NGS workflow is meticulously mapped out to include all essential steps from raw data acquisition to final reporting. The designed workflow encompasses:

- a. Data Acquisition:** Downloading raw NGS data (reads in FASTQ format) using tools like SRA Toolkit.
- b. Quality Control:** Performing quality filtering and adapter trimming using FastQC and Fastp to ensure data integrity.
- c. Alignment and Mapping:** Utilizing alignment tools such as BWT or Bowtie for mapping reads to reference genomes.
- d. Base Quality Recalibration:** Applying GATK for recalibrating base quality scores to improve variant calling accuracy.
- e. Variant Calling:** Identifying mutations through variant calling processes.

2. Pipeline Testing:

Each step of the pipeline undergoes rigorous testing using sample datasets to validate functionality, ensure data integrity, and optimize performance. Automated scripts and shell scripting are employed to facilitate task automation, manage file operations, and execute command-line tools efficiently.



STEP 5.3: Backend Development and Integration of NGS Tools

1. Programming Language Selection:

The backend development leverages a combination of programming languages to harness their respective strengths:

- a. **Python:** Utilized for its extensive libraries that facilitate NGS data analysis and GUI development.
- b. **Shell Scripting:** Used for automating various tasks within the NGS workflow, enhancing efficiency in file management and tool execution.
- c. **Additional Languages:** May be incorporated as needed to support specialized data analysis and processing requirements.

2. Integration of Bioinformatics Tools:

Existing bioinformatics tools and algorithms are seamlessly integrated into the backend to support comprehensive data analysis, visualization, and interpretation of NGS results. This integration ensures that users can perform end-to-end analyses within a unified platform without the need for external tool dependencies.

NGS Pipeline Implementation:

Downloading Raw NGS Data:

Utilizes SRA Toolkit to download raw sequencing reads in FASTQ format from public repositories.



Quality Filtering and Adapter Trimming:

Employs FastQC for initial quality assessment and Fastp for filtering low-quality reads and trimming adapter sequences to ensure high-quality input data.



Alignment and Mapping:

Uses alignment tools like BWT or Bowtie to map the filtered reads to a reference genome, producing aligned sequencing data.



Base Quality Recalibration:

Applies GATK for recalibrating base quality scores, enhancing the accuracy of subsequent variant calling.



Variant Calling:

Identifies genetic variants (mutations) from the recalibrated data, providing insights into genetic differences relevant to precision medicine.



STEP 5.4: GUI/Frontend Development

1. Prototype Development:

An initial prototype of the GUI is developed using suitable development tools and frameworks that support graphical interface design. The prototype serves as a foundational model to visualize workflow steps and gather preliminary user feedback.

2. User Interface Design:

The GUI is designed following best practices in user interface design to ensure it is intuitive, responsive, and accessible. Key features include:

- a. Workflow Visualization:** Clear mapping of NGS workflow steps, allowing users to navigate through data acquisition, analysis, and reporting seamlessly.
- b. Interactive Elements:** Incorporation of interactive components such as buttons, progress bars, and data visualization tools to enhance user engagement.
- c. Customization Options:** Providing users with the ability to configure analysis parameters and customize reports based on specific research needs.



STEP 5.5: Validation and Optimization

1. Performance Validation:

The developed GUI and integrated NGS pipeline are subjected to extensive testing using real NGS datasets. This phase assesses the accuracy, reliability, and efficiency of the workflow. Key performance indicators include:

- a. Data Processing Speed:** Measuring the time taken for each pipeline step to ensure timely analysis.
- b. Accuracy of Results:** Verifying the correctness of variant calling and data interpretations against known benchmarks.
- c. System Reliability:** Ensuring the software operates consistently without crashes or data loss.

2. Optimization Strategies:

Based on validation results, algorithms and workflow processes are optimized to enhance performance. Optimization efforts focus on:

- a. Algorithm Refinement:** Improving the efficiency and accuracy of data processing algorithms.
- b. Resource Management:** Enhancing the software's ability to handle large datasets without compromising performance.

- c. **User Experience Enhancements:** Streamlining processes to reduce user effort and improve overall satisfaction.



STEP 5.6: Documentation and Release
--

1. Comprehensive Documentation:

Detailed documentation is developed to cover all aspects of the project, including step-by-step guides on how to use the software, perform analyses, and interpret results.

2. Final Release Preparation:

The software undergoes final quality assurance checks to ensure all components function as intended. Packaging includes the executable software, documentation, and necessary dependencies to facilitate easy installation and usage by end-users.

3. Release and Deployment:

The finalized software is released to the target audience through appropriate distribution channels. Ongoing support and updates are planned to address any emerging issues, incorporate user feedback, and implement future enhancements.

CHAPTER 6:

CODE

CODE

GitHub Repository Link: <https://github.com/PrarthiKothari/NGSPrecisionMedToolkit>

6.1. reset.py

```
import os
import sys
from tools import run_command, run_command_out

try:
    os.remove('.env')

except:
    print("An error occurred while resetting the environment.")
    sys.exit(1)

try:
    run_command('rm -rf applications')
    # run_command("sudo apt-get remove docker docker-engine docker.io containerd runc")
    # run_command("sudo apt-get autoremove")

    # Stop the Docker service
    run_command_out("sudo systemctl stop docker")

    # Remove Docker packages and configuration files
    run_command_out("sudo apt-get purge docker-ce docker-ce-cli containerd.io -y")

    # Clean up unused dependencies
    run_command_out("sudo apt-get autoremove --purge -y")

    # Remove remaining Docker directories
    run_command_out("sudo rm -rf /etc/docker")
    run_command_out("sudo rm -rf /var/lib/docker")
    run_command_out("sudo rm -rf /var/lib/containerd")

    # # Optionally remove the Docker group
    # run_command_out("sudo groupdel docker")

    # Verify if Docker is removed
    run_command_out("sudo systemctl status docker")

except:
    print("An error occurred while resetting the environment.")
    sys.exit(1)
```


6.2. tools.py

```
import os
import subprocess
import requests
from tqdm import tqdm
import sys

def update_path(path_var, path):
    with open(".env", "r") as f:
        lines = f.readlines()
    for line in lines:
        if path is not None:
            if path_var in line:
                lines[lines.index(line)] = f'{path_var}='{path}'\n"
    with open(".env", "w") as f:
        f.writelines(lines)

def add_path(path_var, path):
    with open(".env", "a") as f:
        if path is not None:
            f.write(f'{path_var}='{path}'\n")

def set_paths(path_var, temppath):
    if not os.path.isfile('.env'):
        try:
            with open('.env', "w") as f:
                f.write("") # Create an empty .env file
        except Exception as e:
            print(f"An error occurred while creating the .env file: {e}")
            sys.exit(1)
    try:
        with open(".env", "r") as f:
            lines = f.read()
        if path_var in lines:
            update_path(path_var, temppath)
        else:
            add_path(path_var, temppath)
    except:
        print("An error occurred while updating the path.")
        sys.exit(1)

def run_command_out(cmd, dir=None):
    cmd = cmd.split()
```

```

# print("Running command:", cmd)
if dir:
    cmd[0] = dir+"/"+cmd[0]
try:
    output = subprocess.check_output(cmd)
    return output.decode("utf-8")
except subprocess.CalledProcessError as e:
    print("An error occurred while running the command:", e.output.decode())
    return cmd

def run_command(cmd, dir=None):
    cmd = cmd.split()
    # print("Running command:", cmd)
    if dir:
        cmd[0] = dir+"/"+cmd[0]
    try:
        subprocess.run(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
        return True
    except subprocess.CalledProcessError as e:
        return False

def download(url, fname, chunk_size=1024):
    headers = {
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.3'
    }
    if os.path.isfile(fname):
        print(f"File '{fname}' already exists.")
    else:
        try:
            resp = requests.get(url, headers=headers, stream=True)
            resp.raise_for_status() # Raise an error for bad responses
            total = int(resp.headers.get('content-length', 0))
            with open(fname, 'wb') as file, tqdm(
                desc=fname,
                total=total,
                unit='iB',
                unit_scale=True,
                unit_divisor=1024,
            ) as bar:
                for data in resp.iter_content(chunk_size=chunk_size):
                    size = file.write(data)
                    bar.update(size)
            print(f"Downloaded '{fname}' successfully.")

```

```

except requests.exceptions.RequestException as e:
    print(f'An error occurred: {e}')
    return None
return fname

```

6.3. setup.py

```

import os
import subprocess
import sys
import time
from tools import run_command, run_command_out, set_paths
from dotenv import load_dotenv

load_dotenv()
app_dir = os.getenv("APP_DIR")
data_dir = os.getenv("DATA_DIR")

def setup_sudo(sudo_password):
    if sudo_password is None:
        print("SUDO_PASSWORD is not set in .env file.")
    else:
        try:
            commands = [ "sudo -S apt-get update" ]
            for command in commands:
                try:
                    process = subprocess.Popen(command.split(), stdin=subprocess.PIPE,
stdout=subprocess.PIPE, stderr=subprocess.PIPE)
                    output, error = process.communicate(input=(sudo_password + '\n').encode())
                    print(output.decode("utf-8"))
                    if error:
                        print("Error:", error.decode("utf-8"))
                except Exception as e:
                    print("Failed to run command:", command, "Error:", str(e))
            return True
        except Exception as e:
            print("An error occurred while setting up sudo:", str(e))
            print("Please make sure the password is correct.")
            sys.exit(1)

def setup_deps(sudo=True):
    try:
        if sudo:
            run_command_out("sudo apt-get install -y wget")

```

```

run_command_out("sudo apt-get install -y python3-pip")
run_command_out("sudo apt-get install -y default-jdk")
run_command_out("sudo apt-get install -y default-jre")
run_command_out("sudo apt-get install -y unzip")

# for bwa
run_command_out("sudo apt-get install -y build-essential")
run_command_out("sudo apt-get install -y bwa")
run_command_out("gcc --version")
run_command_out("sudo apt-get install -y zlib1g-dev")

return True
else:
    run_command_out("apt-get install -y wget")
    run_command_out("apt-get install -y python3-pip")
    run_command_out("apt-get install -y default-jdk")
    run_command_out("apt-get install -y default-jre")
    run_command_out("apt-get install -y unzip")

# for bwa
run_command_out("apt-get install -y build-essential")
run_command_out("apt-get install -y bwa")
run_command_out("gcc --version")
run_command_out("apt-get install -y zlib1g-dev")

return True
except Exception as e:
    print("An error occurred while setting up dependencies:", str(e))
    return False

def setup_sra(app_dir):
    sra_path = f"{app_dir}/sratoolkit/bin"
    try:
        run_command_out("vdb-dump --help", dir=sra_path)
    except:
        run_command_out(f"wget -O {app_dir}/sratoolkit.tar.gz https://ftp-
trace.ncbi.nlm.nih.gov/sra/sdk/3.1.1/sratoolkit.3.1.1-ubuntu64.tar.gz")
        run_command_out(f"tar -xzf {app_dir}/sratoolkit.tar.gz -C {app_dir}")
        run_command_out(f"mv {app_dir}/sratoolkit.3.1.1-ubuntu64 {app_dir}/sratoolkit")
        run_command_out(f"rm {app_dir}/sratoolkit.tar.gz")
    return sra_path

def setup_fastqc(app_dir):
    fastqc_path = f"{app_dir}/fastqc"

```

```

try:
    run_command_out("fastqc --help", dir=fastqc_path)
except:
    run_command_out(f'wget -O {app_dir}/fastqc.zip
https://www.bioinformatics.babraham.ac.uk/projects/fastqc/fastqc_v0.12.1.zip")
    run_command_out(f'unzip {app_dir}/fastqc.zip -d {app_dir}')
    run_command_out(f'rm {app_dir}/fastqc.zip')
    run_command_out(f'mv {app_dir}/FastQC {app_dir}/fastqc-temp')
    run_command_out(f'mv {app_dir}/fastqc-temp {app_dir}/fastqc')
return fastqc_path

def setup_fastp(app_dir):
    fastp_path = f'{app_dir}/fastp'
    os.makedirs(fastp_path, exist_ok=True)
    try:
        run_command_out("fastp --help", dir=fastp_path)
    except:
        run_command_out(f'wget -O {fastp_path}/fastp http://opengene.org/fastp/fastp")
        run_command_out(f'chmod a+x {fastp_path}/fastp")
    return fastp_path

def setup_bwa(app_dir):
    bwa_path = 'bwa'
    try:
        run_command_out("bwa")
    except Exception as e:
        print("BWA is not installed. Proceeding with installation...")

        try:
            try:
                run_command_out(f'sudo apt-get install -y bwa")
            except:
                run_command_out(f'apt-get install -y bwa")
        except:
            print("Automatic BWA installation failed. Please install BWA manually")

    return bwa_path

def setup_docker(app_dir):
    """Sets up Docker in the specified application directory."""
    docker_path = f'{app_dir}/docker'
    os.makedirs(docker_path, exist_ok=True)

```

```

try:
    run_command_out("docker -V")
    print("Docker is already installed.")

except Exception as e:
    print("Docker is not installed. Proceeding with installation...")

    # Install prerequisites
    run_command_out("sudo apt-get update")
    run_command_out("sudo apt-get install -y ca-certificates curl")
    run_command_out("sudo install -m 0755 -d /etc/apt/keyrings")

    # Add Docker's official GPG key
    run_command_out("sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc")
    run_command_out("sudo chmod a+r /etc/apt/keyrings/docker.asc")

    # Get architecture
    arch = subprocess.check_output("dpkg --print-architecture", shell=True).decode().strip()

    # Get Ubuntu codename
    codename = subprocess.check_output("lsb_release -cs", shell=True).decode().strip()

    # Create Docker source list entry
    docker_source = f'deb [arch={arch}] signed-by=/etc/apt/keyrings/docker.asc [
https://download.docker.com/linux/ubuntu {codename} stable'
    subprocess.run(f'echo "{docker_source}" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null', shell=True, check=True)

    run_command_out("sudo apt-get update -y")

    # Install Docker
    run_command_out("sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin")

    # Start and enable Docker service
    run_command_out("sudo systemctl start docker")
    run_command_out("sudo systemctl enable docker")

    # Check status of Docker service
    run_command_out("sudo systemctl status docker")

return docker_path

```

```

def setup_bcftools(app_dir):
    bcftools_path = "bcftools"
    try:
        run_command_out(f"bcftools")
    except:
        print("BCFtools is not installed. Proceeding with installation...")
        try:
            try:
                run_command_out(f"sudo apt-get install -y bcftools")
            except:
                run_command_out(f"apt-get install -y bcftools")
        except:
            print("Automatic BCFtools installation failed. Please install BCFtools manually")

    return bcftools_path

def setup_samtools(app_dir):
    samtools_path = "samtools"
    try:
        run_command_out(f"samtools")
    except:
        print("samtools is not installed. Proceeding with installation...")
        try:
            try:
                run_command_out(f"sudo apt-get install -y samtools")
            except:
                run_command_out(f"apt-get install -y samtools")
        except:
            print("Automatic samtools installation failed. Please install samtools manually")

    return samtools_path

# def setup_gatk(app_dir):
#     gatk_path = f"{app_dir}/gatk"
#     os.makedirs(gatk_path, exist_ok=True)
#     try:
#         run_command_out("gatk --help", dir=gatk_path)
#     except:
#         run_command_out(f"wget -O {gatk_path}/gatk.zip")

def main(sudo_password=None):
    while sudo_password is None:
        try:
            sudo_password = os.getenv("SUDO_PASSWORD")

```

```

    if sudo_password is None:
        raise Exception("SUDO_PASSWORD is not set in .env file.")
except:
    sudopass = input("Please enter the sudo password press Enter to continue: ")
    set_paths("SUDO_PASSWORD", sudopass.strip())
    set_paths("APP_DIR", 'applications')
    set_paths("DATA_DIR", 'data')
    print("SUDO_PASSWORD set successfully.")
    print("Please run the setup script again.")
    sys.exit(0)
os.makedirs(app_dir, exist_ok=True)
os.makedirs(data_dir, exist_ok=True)
try:
    if setup_sudo(sudo_password):
        setup_deps(sudo=True)
        sra_path = setup_sra(app_dir)
        set_paths("SRAToolKIT_PATH", sra_path)
        fastqc_path = setup_fastqc(app_dir)
        set_paths("FASTQC_PATH", fastqc_path)
        fastp_path = setup_fastp(app_dir)
        set_paths("FASTP_PATH", fastp_path)
        bwa_path = setup_bwa(app_dir)
        set_paths("BWA_PATH", bwa_path)
        bcftools_path = setup_bcftools(app_dir)
        set_paths("BCFTOOLS_PATH", bcftools_path)
        samtools_path = setup_samtools(app_dir)
        set_paths("SAMTOOLS_PATH", samtools_path)
        docker_path = setup_docker(app_dir)
        set_paths("DOCKER_PATH", docker_path)
        print(f"Docker has been set up at: {docker_path}")

        # GATK
        gatk_path = setup_gatk(app_dir)
        set_paths("GATK_PATH", gatk_path)

        print("Setup completed successfully.")
    else:
        setup_deps(sudo=False)
        sra_path = setup_sra(app_dir)
        set_paths("SRAToolKIT_PATH", sra_path)
        fastqc_path = setup_fastqc(app_dir)
        set_paths("FASTQC_PATH", fastqc_path)
        fastp_path = setup_fastp(app_dir)

```



```

    set_paths("FASTP_PATH", fastp_path)
    bwa_path = setup_bwa(app_dir)
    set_paths("BWA_PATH", bwa_path)
    bcftools_path = setup_bcftools(app_dir)
    set_paths("BCFTOOLS_PATH", bcftools_path)
    samtools_path = setup_samtools(app_dir)
    set_paths("SAMTOOLS_PATH", samtools_path)
    docker_path = setup_docker(app_dir)
    set_paths("DOCKER_PATH", docker_path)
    print(f'Docker has been set up at: {docker_path}')

    # GATK
    #gatk_path = setup_gatk(app_dir)
    #set_paths("GATK_PATH", gatk_path)

    print("Setup completed successfully.")
except Exception as e:
    print(e)

if __name__ == "__main__":
    system_os = sys.platform
    if system_os != 'linux':
        print("This application is specifically built for Linux systems.")
        print("Please install WSL on Windows or use VirtualBox to run this application.")
        for i in range(30):
            print("Closing application in", 30-i, "seconds...", end="\r")
            time.sleep(1)
        sys.exit(1)
    main()

```

6.4. get_fastq.py

```

import os
import sys
from dotenv import load_dotenv
from tools import run_command_out, set_paths

load_dotenv()
app_dir = os.getenv("APP_DIR")
data_dir = os.getenv("DATA_DIR")
sratoolkit_path = os.getenv("SRATOOLKIT_PATH")

def cmd_for_info(accession: str):
    command = (f'vdb-dump {accession} --info')

```

```

return command

def cmd_to_download(accession: str,
                    alignment_filter: bool,
                    compressed: bool,
                    skip_technical: bool,
                    remove_adapter: bool,
                    spot_group: bool,
                    alignment_filter_type: str = None,
                    min_reads: int = None,
                    max_reads: int = None,
                    ar_specific: str = None,
                    ar_start: int = None,
                    ar_end: int = None,
                    member: str = None):
    command = ['fastq-dump', accession, '--split-3', '--outdir', f'data/{accession}']
    if compressed:
        command.append('--gzip')
    else:
        command = command
    if alignment_filter:
        if alignment_filter_type == "split-spot":
            command.remove('--split-3')
            command.append('--split-spot')
        elif alignment_filter_type == "aligned":
            command.remove('--split-3')
            command.append('--aligned')
        elif alignment_filter_type == "unaligned":
            command.remove('--split-3')
            command.append('--unaligned')
        elif alignment_filter_type == "aligned-region":
            command.remove('--split-3')
            command.append('--aligned-region')
            command.append(f'{ar_specific}:{ar_start}-{ar_end}')
        elif alignment_filter_type == "matepair-distance":
            command.remove('--split-3')
            command.append('--matepair-distance')
            command.append(f'{ar_start}-{ar_end}')
    else:
        command = command
    else:
        command = command
    if skip_technical:
        command.append('--skip-technical')

```

```

else:
    command = command
if min_reads:
    command.append(f'--minSpotId')
    command.append(str(min_reads))
else:
    command = command
if max_reads:
    command.append(f'--maxSpotId')
    command.append(str(max_reads))
else:
    command = command
if remove_adapter:
    command.append('--clip')
else:
    command = command
if spot_group:
    command.append('--spot-group')
    command.append(member)
else:
    command = command
cmd = ' '.join(command)
return cmd

def list_files_in_directory(directory):
    try:
        # List all files in the specified directory
        files = [f for f in os.listdir(directory) if os.path.isfile(os.path.join(directory, f))]
        return files
    except FileNotFoundError:
        print(f'The directory '{directory}' does not exist.")
        return []
    except Exception as e:
        print(f'An error occurred: {e}")
        return []

def main(accession: str, alignment_filter_type: str,
        alignment_filter: bool, compressed: bool, skip_technical: bool, remove_adapter: bool,
        spot_group: bool,
        min_reads: int = None, max_reads: int = None, ar_specific: str = None, ar_start: int =
        None, ar_end: int = None, member: str = None):
    try:
        run_command_out("vdb-dump --help", dir=sratoolkit_path)
    except Exception as e:

```

```

    print(f'An error occurred: {e}')
    print("Setup Incomplete. Please run the setup script again.")
    sys.exit(1)
    working_dir = os.path.join(data_dir, accession)
    set_paths("WORKING_DIR", working_dir)
    os.makedirs(name=working_dir, mode=0o777, exist_ok=True)
    get_data = cmd_for_info(accession=accession)
    download = cmd_to_download(accession, alignment_filter, compressed, skip_technical,
remove_adapter, spot_group, alignment_filter_type, min_reads, max_reads, ar_specific,
ar_start, ar_end, member)
    print("Fetching data")
    data = run_command_out(get_data, dir=sratoolkit_path)
    print("Downloading Fastq files")
    download_status = run_command_out(download, dir=sratoolkit_path)
    files = []
    for file in list_files_in_directory(working_dir):
        file = os.path.join(working_dir, file)
        files.append(file)
    return data, download_status, files

if __name__ == "__main__":
    accession = "ERR11468775"
    compressed = True
    alignment_filter = False
    alignment_filter_type = None
    skip_technical = False
    remove_adapter = False
    spot_group = False
    min_reads = None
    max_reads = 100
    ar_specific = None
    ar_start = None
    ar_end = None
    member = None
    data, download_status, files = main(accession=accession,
alignment_filter=alignment_filter, alignment_filter_type=alignment_filter_type,
compressed=compressed,
    skip_technical=skip_technical, remove_adapter=remove_adapter,
spot_group=spot_group, min_reads=min_reads, max_reads=max_reads,
    ar_specific=ar_specific, ar_start=ar_start, ar_end=ar_end, member=member)
    print(f'{accession} DATA:\n'+data)
    print("DOWNLOAD STATUS:\n"+download_status)
    for file in files:
        print("SRA FILES: "+file)

```

6.5. quality_check.py

```
import os, sys
import tarfile
from dotenv import load_dotenv
from tools import run_command_out, set_paths

load_dotenv()
fastqc_path = os.getenv("FASTQC_PATH")
working_dir = os.getenv("WORKING_DIR")

def fastqc_check(file, working_dir):
    file_path = os.path.join(working_dir, file)
    if os.path.isfile(file_path):
        if any(ext in file for ext in [".fastq", ".sam", ".bam"]):
            try:
                output_dir = os.path.join(working_dir, 'fastqc_reports')
                os.makedirs(output_dir, exist_ok=True)
                run_command_out(f'fastqc {file_path} --outdir {output_dir}', dir=fastqc_path)
            except Exception as e:
                print(f'An error occurred while running the command: {e}')
        else:
            print(f'The file format of {file} is not supported.')

def main(working_dir):
    fastqc_data_dir = os.path.join(working_dir, 'fastqc_reports')
    set_paths("FASTQC_DATA_DIR", fastqc_data_dir)
    files = os.listdir(working_dir)
    for file in files:
        fastqc_check(file, working_dir)
    files = os.listdir(fastqc_data_dir)
    reports = []
    for file in files:
        if 'html' in file:
            report = os.path.join(fastqc_data_dir, file)
            print(f'FastQC report for {file.strip('.html').replace('_fastqc', '.fastq.gz')} is ready at {report}')
            reports.append(file)
    return reports

if __name__ == '__main__':
    reports = main(working_dir)
    for report in reports:
        print(report)
```

6.6. trimadapters.py

```
import os
from dotenv import load_dotenv
from tools import run_command_out, set_paths

load_dotenv()
working_dir = os.getenv("WORKING_DIR")
fastp_path = os.getenv("FASTP_PATH")

def trim_adapters(fastq_file_1, fastq_file_2):
    output_dir = os.path.join(working_dir, 'fastp_reports')
    os.makedirs(output_dir, exist_ok=True)

    output_file_1 = os.path.join(working_dir,
    f'{os.path.basename(fastq_file_1).replace('.fastq.gz', '_trimmed.fastq.gz')}')
    output_file_2 = os.path.join(working_dir,
    f'{os.path.basename(fastq_file_2).replace('.fastq.gz', '_trimmed.fastq.gz')}')

    # Construct the command for fastp
    command = (
        f'{fastp_path}/fastp"
        f" -i {fastq_file_1}"
        f" -o {output_file_1}"
        f" -I {fastq_file_2}"
        f" -O {output_file_2}"
        f" --detect_adapter_for_pe"
        f" -f 10 -g -l 50"
        f" -c -h {output_dir}/ERR11468775_fastp.html"
        f" -w 10"
    )

    try:
        run_command_out(command)
        print(f"Trimmed files created: {output_file_1}, {output_file_2}")
    except Exception as e:
        print(f"Error trimming adapters: {e}")

    return output_file_1, output_file_2

def main(working_dir):
    fastp_data_dir = os.path.join(working_dir, 'fastp_reports')
    set_paths("FASTP_DATA_DIR", fastp_data_dir)
```

```

fastq_file_1 = os.path.join(working_dir, "ERR11468775_1.fastq.gz")
fastq_file_2 = os.path.join(working_dir, "ERR11468775_2.fastq.gz")

trimmed_files = []
output_file_1, output_file_2 = trim_adapters(fastq_file_1, fastq_file_2)
trimmed_files.append(output_file_1)
trimmed_files.append(output_file_2)

return trimmed_files

if __name__ == '__main__':
    trimmed_files = main(working_dir)
    data_dir = os.path.join(working_dir, 'trimmed')
    os.makedirs(data_dir, exist_ok=True)
    for file in trimmed_files:
        trimmed_path = os.path.join(data_dir, os.path.basename(file))
        os.rename(file, trimmed_path)
        print(f'Moved {file} to {trimmed_path}')

```

6.7. ref_genome.py

```

import os
import tarfile
from dotenv import load_dotenv
from tools import run_command_out

load_dotenv()
app_dir = os.getenv("APP_DIR")
data_dir = os.getenv("DATA_DIR")

def download_ref_genome():
    ref_genome_path = os.path.join(data_dir, "ref_genome")
    os.makedirs(ref_genome_path, exist_ok=True)

    #try:
    ref_genome_file = os.path.join(ref_genome_path, "chr18.fa")
    if os.path.exists(ref_genome_file):
        print(f'Reference genome already exists.')
        return

    # except Exception as e:
    #     print(f'An error occurred while checking for reference genome: {e}')

    print(f'Downloading reference genome...')

```

```

download_url =
"https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr18.fa.gz"
run_command_out(f"wget -O {ref_genome_path}/ref_genome.gz {download_url}")
run_command_out(f"gunzip {ref_genome_path}/ref_genome.gz")
run_command_out(f"mv {ref_genome_path}/ref_genome {ref_genome_path}/chr18.fa")
print(f"Reference genome downloaded and unzipped successfully.")

if __name__ == '__main__':
    download_ref_genome()

```

6.8. bwa_index_map.py

```

import os
from dotenv import load_dotenv
from tools import run_command_out, set_paths

load_dotenv()
working_dir = os.getenv("WORKING_DIR")
bwa_path = "bwa"
data_dir = os.getenv("DATA_DIR")

def bwa_index_map(trimmed_fastq_file_1, trimmed_fastq_file_2, ref_genome):
    bwa_data_dir = os.path.join(working_dir, 'bwa_index_map')
    os.makedirs(bwa_data_dir, exist_ok=True)
    chr18_ref = os.path.join(bwa_data_dir, 'chr18_ref')

    print("Indexing reference genome...")
    run_command_out(f'{bwa_path} index -a bwtsw -p {chr18_ref} {ref_genome}')

    print("Mapping trimmed FASTQ files...")
    sam_output_file = os.path.join(bwa_data_dir, "sample1.sam")
    run_command_out(f'{bwa_path} mem {chr18_ref} {trimmed_fastq_file_1}
{trimmed_fastq_file_2} -t 10 -o {sam_output_file}')

    print(f'Mapping completed. Output SAM file: {sam_output_file}')
    return sam_output_file

def main(working_dir):
    bwa_data_dir = os.path.join(working_dir, 'bwa_index_map')
    set_paths("BWA_DATA_DIR", bwa_data_dir)

    trimmed_fastq_file_1 = os.path.join(working_dir, "trimmed",
"ERR11468775_1_trimmed.fastq.gz")

```



```

    trimmed_fastq_file_2 = os.path.join(working_dir, "trimmed",
"ERR11468775_2_trimmed.fastq.gz")
    ref_genome = os.path.join(data_dir, "ref_genome", "chr18.fa")

    samfile_1 = bwa_index_map(trimmed_fastq_file_1, trimmed_fastq_file_2, ref_genome)

    return samfile_1

if __name__ == '__main__':
    mapped_files = main(working_dir)

    print(f'Mapped files: {mapped_files}')

```

6.9. sbgatk_analysis.py

```

import os
import subprocess
import time
from dotenv import load_dotenv

load_dotenv()
data_dir = os.getenv("DATA_DIR")
working_dir = os.getenv("WORKING_DIR")

if not data_dir or not working_dir:
    raise EnvironmentError("DATA_DIR and WORKING_DIR must be set in your
environment.")

def run_command(command, error_message):
    try:
        print(f"\n[Running] {command}")
        result = subprocess.run(command, shell=True, check=True,
                                stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                                text=True)
        if result.stdout:
            print(result.stdout)
        if result.stderr:
            print(result.stderr)
    except subprocess.CalledProcessError as e:
        print(f"\n[Error] {error_message}")
        print(f'Command: {command}')
        print(f'Error output: {e.stderr}')
        raise

```

```

def sbgatk_analysis(samfile):
    # Create the analysis output folder (inside DATA_DIR)
    analysis_dir = os.path.join(data_dir, 'SBGATK_ANALYSIS')
    os.makedirs(analysis_dir, exist_ok=True)
    print(f'Analysis output will be stored in: {analysis_dir}')

    # Check for the required reference file (chr18.fa)
    chr18_ref = os.path.join(data_dir, "chr18.fa")
    if not os.path.exists(chr18_ref):
        raise FileNotFoundError(f'Reference file {chr18_ref} not found. Please place chr18.fa
in {data_dir}.')

    # Download dbSNP VCF and index
    dbsnp_vcf = os.path.join(analysis_dir, "Homo_sapiens_assembly38.dbsnp138.vcf")
    dbsnp_vcf_idx = os.path.join(analysis_dir,
"Homo_sapiens_assembly38.dbsnp138.vcf.idx")
    try:
        if not os.path.exists(dbsnp_vcf):
            run_command(f'wget https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/Homo_sapiens_assembly38.dbsnp138.vcf -O {dbsnp_vcf}',
                "Failed to download Homo_sapiens_assembly38.dbsnp138.vcf")
        else:
            print("dbSNP VCF already exists.")
        if not os.path.exists(dbsnp_vcf_idx):
            run_command(f'wget https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/Homo_sapiens_assembly38.dbsnp138.vcf.idx -O
{dbsnp_vcf_idx}',
                "Failed to download Homo_sapiens_assembly38.dbsnp138.vcf.idx")
        else:
            print("dbSNP VCF index already exists.")
    except Exception as e:
        print(e)
        return

    # Download hapmap files
    hapmap_vcf = os.path.join(analysis_dir, "hapmap_3.3.hg38.vcf.gz")
    hapmap_vcf_tbi = os.path.join(analysis_dir, "hapmap_3.3.hg38.vcf.gz.tbi")
    try:
        if not os.path.exists(hapmap_vcf):
            run_command(f'wget https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz -O {hapmap_vcf}',
                "Failed to download hapmap_3.3.hg38.vcf.gz")
        else:
            print("hapmap VCF already exists.")

```

```

    if not os.path.exists(hapmap_vcf_tbi):
        run_command(f'wget https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz.tbi -O {hapmap_vcf_tbi}',
                    "Failed to download hapmap_3.3.hg38.vcf.gz.tbi")
    else:
        print("hapmap VCF index already exists.")
except Exception as e:
    print(e)
    return

sample_bam = os.path.join(analysis_dir, "sample1.bam")
if not os.path.exists(sample_bam):
    if not os.path.exists(samfile):
        raise FileNotFoundError(f"Input SAM file {samfile} not found.")
    try:
        run_command(f'samtools view -bo {sample_bam} {samfile}',
                    "Failed to convert SAM to BAM")
    except Exception as e:
        print(e)
        return
else:
    print("sample1.bam already exists.")

script_path = os.path.join(analysis_dir, "run_analysis.sh")
try:
    with open(script_path, "w") as f:
        f.write("#!/bin/bash\n")
        f.write("set -e\n")
        # Change directory to where the analysis files are mounted (inside the container
/data/SBGATK_ANALYSIS)
        f.write("cd /data/SBGATK_ANALYSIS\n\n")
        f.write("echo 'Running AddOrReplaceReadGroups'\n")
        f.write("gatk AddOrReplaceReadGroups -I sample1.bam -O sample1_withRG.bam -
ID 1 -LB lib1 -PL ILLUMINA -PU unit1 -SM sample1\n\n")

        f.write("echo 'Running SortSam'\n")
        f.write("gatk SortSam -I sample1_withRG.bam -O sorted_sample1.bam -SO
coordinate\n\n")

        f.write("echo 'Running samtools flagstat'\n")
        f.write("samtools flagstat sorted_sample1.bam > sorted_sample1_flagstat.txt\n\n")

        f.write("echo 'Running MarkDuplicates'\n")

```

```

f.write("gatk MarkDuplicates -I sorted_sample1.bam -O markedDups.bam -M
metrics_duplicates.txt\n\n")

f.write("echo 'Listing current files'\n")
f.write("ls -l\n\n")

f.write("echo 'Creating sequence dictionary (if not already present)'\n")
f.write("if [ ! -f chr18.dict ]; then gatk CreateSequenceDictionary -R chr18.fa; fi\n\n")

f.write("echo 'Indexing chr18.fa with samtools faidx'\n")
f.write("samtools faidx chr18.fa\n\n")

f.write("echo 'Running BaseRecalibrator'\n")
f.write("gatk BaseRecalibrator -I markedDups.bam -R chr18.fa --known-sites
Homo_sapiens_assembly38.dbsnp138.vcf -O sample1_recal_data.table\n\n")

f.write("echo 'Applying BQSR'\n")
f.write("gatk ApplyBQSR -R chr18.fa -I markedDups.bam --bqsr-recal-file
sample1_recal_data.table -O sample1_recal.bam\n\n")

f.write("echo 'Indexing recalibrated BAM'\n")
f.write("samtools index sample1_recal.bam\n\n")

f.write("echo 'Running Mutect2'\n")
f.write("gatk Mutect2 -I sample1_recal.bam -R chr18.fa -O sample1.vcf.gz\n\n")

f.write("echo 'Filtering Mutect calls'\n")
f.write("gatk FilterMutectCalls -R chr18.fa -V sample1.vcf.gz -O
filtered_sample1.vcf.gz\n\n")

f.write("echo 'Extracting indels with bcftools'\n")
f.write("bcftools view --types indels sample1.vcf.gz > sample1_indels.vcf\n\n")

f.write("echo 'Extracting SNPs with bcftools'\n")
f.write("bcftools view --types snps sample1.vcf.gz > sample1_snps.vcf\n\n")

f.write("echo 'Filtering variants with QUAL>50'\n")
f.write("bcftools filter -i '%QUAL>50' sample1.vcf.gz -o sample1_filtered.vcf\n\n")

f.write("echo 'Running HaplotypeCaller for germline GVCF'\n")
f.write("gatk HaplotypeCaller -R chr18.fa -I sample1_recal.bam -O
sample1_germline.g.vcf.gz -ERC GVCF\n\n")

```

```

        f.write("if [ ! -f sample2_germline.g.vcf.gz ]; then cp sample1_germline.g.vcf.gz
sample2_germline.g.vcf.gz; fi\n")
        f.write("if [ ! -f sample3_germline.g.vcf.gz ]; then cp sample1_germline.g.vcf.gz
sample3_germline.g.vcf.gz; fi\n\n")

        f.write("echo 'Combining gVCF files with GenomicsDBImport'\n")
        f.write("gatk GenomicsDBImport -V sample1_germline.g.vcf.gz -V
sample2_germline.g.vcf.gz -V sample3_germline.g.vcf.gz --genomicsdb-workspace-path
test_db --intervals chr18\n\n")

        f.write("echo 'Genotyping GVCFs'\n")
        f.write("gatk GenotypeGVCFs -R chr18.fa -V gendb://test_db -O try_new.vcf.gz\n\n")

        f.write("echo 'Extracting SNPs from combined VCF'\n")
        f.write("bcftools view --types snps try_new.vcf.gz > try_new_snps.vcf\n\n")

        f.write("echo 'GATK Analysis Completed'\n")
        os.chmod(script_path, 0o755)
        print(f'Bash script for analysis created at: {script_path}')
    except Exception as e:
        print(f'Error creating bash script: {e}')
    return

    try:
        run_command("docker pull broadinstitute/gatk:latest", "Failed to pull the GATK Docker
image")
    except Exception as e:
        print(e)
    return

    docker_cmd = (f'docker run --rm -v {data_dir}:/data broadinstitute/gatk:latest "
        f'/bin/bash /data/SBGATK_ANALYSIS/run_analysis.sh')

    try:
        run_command(docker_cmd, "Failed to run the analysis script in the Docker container")
    except Exception as e:
        print(e)
    return

    print("\nGATK analysis completed successfully. Check the SBGATK_ANALYSIS
directory for output files.")

def main():
    samfile = os.path.join(working_dir, "bwa_index_map", "sample1.sam")
    if not os.path.exists(samfile):

```

```

        raise FileNotFoundError(f'Input SAM file {samfile} not found. Please check your
WORKING_DIR path.")
    sbgatk_analysis(samfile)

if __name__ == '__main__':
    main()

```

6.10. app.py

```

import os
import sys
import subprocess
import requests
from tqdm import tqdm
import shutil
import time
import tarfile
import gradio as gr
from dotenv import load_dotenv

def update_path(path_var, path):
    with open(".env", "r") as f:
        lines = f.readlines()
    for i, line in enumerate(lines):
        if path is not None and path_var in line:
            lines[i] = f'{path_var}={path}\n'
    with open(".env", "w") as f:
        f.writelines(lines)

def add_path(path_var, path):
    with open(".env", "a") as f:
        if path is not None:
            f.write(f'{path_var}={path}\n')

def set_paths(path_var, temppath):
    if not os.path.isfile('.env'):
        try:
            with open('.env', "w") as f:
                f.write("")
        except Exception as e:
            print(f'Error creating .env file: {e}')
            sys.exit(1)
    try:
        with open(".env", "r") as f:

```

```

        content = f.read()
    if path_var in content:
        update_path(path_var, temppath)
    else:
        add_path(path_var, temppath)
except Exception as e:
    print("Error updating path:", e)
    sys.exit(1)

def run_command_out(cmd, dir=None):
    cmd_list = cmd.split()
    if dir is not None:
        # If the command is not an absolute path, prepend the directory.
        if not os.path.isabs(cmd_list[0]):
            cmd_list[0] = os.path.join(dir, cmd_list[0])
    try:
        output = subprocess.check_output(cmd_list)
        return output.decode("utf-8")
    except subprocess.CalledProcessError as e:
        print("Error running command:", e.output.decode())
        return cmd

def run_command(cmd, dir=None):
    cmd_list = cmd.split()
    if dir is not None:
        if not os.path.isabs(cmd_list[0]):
            cmd_list[0] = os.path.join(dir, cmd_list[0])
    try:
        subprocess.run(cmd_list, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
        return True
    except subprocess.CalledProcessError as e:
        return False

def download(url, fname, chunk_size=1024):
    headers = {
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.3'
    }
    if os.path.isfile(fname):
        print(f'File '{fname}' already exists.')
    else:
        try:
            resp = requests.get(url, headers=headers, stream=True)

```

```

        resp.raise_for_status()
        total = int(resp.headers.get('content-length', 0))
        with open(fname, 'wb') as file, tqdm(desc=fname, total=total, unit='iB',
unit_scale=True, unit_divisor=1024) as bar:
            for data in resp.iter_content(chunk_size=chunk_size):
                size = file.write(data)
                bar.update(size)
            print(f'Downloaded '{fname}' successfully.")
    except requests.exceptions.RequestException as e:
        print(f'Download error: {e}")
    return None
return fname

def install_sra():
    print("SRA Toolkit not found. Downloading now...")
    fname = "sratoolkit.tar.gz"
    file = download(url="https://ftp-trace.ncbi.nlm.nih.gov/sra/sdk/3.1.1/sratoolkit.3.1.1-
ubuntu64.tar.gz", fname=fname)
    if not file:
        print("Download failed.")
        return
    print("SRA Toolkit downloaded.")
    with tarfile.open(file, "r:gz") as tar:
        tar.extractall()
    print("SRA Toolkit extracted.")
    extracted_dir = "sratoolkit.3.1.1-ubuntu64"
    os.remove(fname)
    if os.path.exists(extracted_dir):
        os.rename(extracted_dir, "sratoolkit")
        print("SRA Toolkit installed.")
    else:
        print(f'Error: '{extracted_dir}' not found after extraction.")

load_dotenv()
app_dir = os.getenv("APP_DIR", "applications")
data_dir = os.getenv("DATA_DIR", "data")
sratoolkit_path = os.getenv("SRATOOLKIT_PATH")
fastqc_path = os.getenv("FASTQC_PATH")
fastp_path = os.getenv("FASTP_PATH")

def setup_sudo(sudo_password):
    if sudo_password is None:
        return "SUDO_PASSWORD is not set in .env file."

```



```

try:
    commands = ["sudo -S apt-get update"]
    output_log = ""
    for command in commands:
        proc = subprocess.Popen(command.split(), stdin=subprocess.PIPE,
stdout=subprocess.PIPE, stderr=subprocess.PIPE)
        output, error = proc.communicate(input=(sudo_password + '\n').encode())
        output_log += output.decode("utf-8")
        if error:
            output_log += "Error: " + error.decode("utf-8") + "\n"
    return output_log
except Exception as e:
    return f"Error setting up sudo: {e}"

def setup_deps(sudo=True):
    try:
        if sudo:
            run_command_out("sudo apt-get install -y wget")
            run_command_out("sudo apt-get install -y python3-pip")
            run_command_out("sudo apt-get install -y default-jdk")
            run_command_out("sudo apt-get install -y default-jre")
            run_command_out("sudo apt-get install -y unzip")
            run_command_out("sudo apt-get install -y build-essential")
            run_command_out("sudo apt-get install -y bwa")
            run_command_out("gcc --version")
            run_command_out("sudo apt-get install -y zlib1g-dev")
            return "Dependencies installed with sudo."
        else:
            run_command_out("apt-get install -y wget")
            run_command_out("apt-get install -y python3-pip")
            run_command_out("apt-get install -y default-jdk")
            run_command_out("apt-get install -y default-jre")
            run_command_out("apt-get install -y unzip")
            run_command_out("apt-get install -y build-essential")
            run_command_out("apt-get install -y bwa")
            run_command_out("gcc --version")
            run_command_out("apt-get install -y zlib1g-dev")
            return "Dependencies installed without sudo."
    except Exception as e:
        return f"Error installing dependencies: {e}"

def setup_sra(app_dir):
    sra_path_local = f"{app_dir}/sratoolkit/bin"

```

```

try:
    run_command_out("vdb-dump --help", dir=sra_path_local)
except Exception:
    run_command_out(f"wget -O {app_dir}/sratoolkit.tar.gz https://ftp-
trace.ncbi.nlm.nih.gov/sra/sdk/3.1.1/sratoolkit.3.1.1-ubuntu64.tar.gz")
    run_command_out(f"tar -xzf {app_dir}/sratoolkit.tar.gz -C {app_dir}")
    run_command_out(f"mv {app_dir}/sratoolkit.3.1.1-ubuntu64 {app_dir}/sratoolkit")
    run_command_out(f"rm {app_dir}/sratoolkit.tar.gz")
return f"{app_dir}/sratoolkit/bin"

def setup_fastqc(app_dir):
    fastqc_path_local = f"{app_dir}/fastqc"
    try:
        run_command_out("fastqc --help", dir=fastqc_path_local)
    except Exception:
        run_command_out(f"wget -O {app_dir}/fastqc.zip
https://www.bioinformatics.babraham.ac.uk/projects/fastqc/fastqc_v0.12.1.zip")
        run_command_out(f"unzip {app_dir}/fastqc.zip -d {app_dir}")
        run_command_out(f"rm {app_dir}/fastqc.zip")
        run_command_out(f"mv {app_dir}/FastQC {app_dir}/fastqc-temp")
        run_command_out(f"mv {app_dir}/fastqc-temp {app_dir}/fastqc")
    return f"{app_dir}/fastqc"

def setup_fastp(app_dir):
    fastp_path_local = f"{app_dir}/fastp"
    os.makedirs(fastp_path_local, exist_ok=True)
    try:
        run_command_out("fastp --help", dir=fastp_path_local)
    except Exception:
        run_command_out(f"wget -O {fastp_path_local}/fastp http://opengene.org/fastp/fastp")
        run_command_out(f"chmod a+x {fastp_path_local}/fastp")
    return fastp_path_local

def run_setup(sudo_password):
    os.makedirs(app_dir, exist_ok=True)
    os.makedirs(data_dir, exist_ok=True)
    log = ""
    log += setup_sudo(sudo_password) + "\n"
    log += setup_deps(sudo=True) + "\n"
    sra_p = setup_sra(app_dir)
    set_paths("SRATOOLKIT_PATH", sra_p)
    global sratoolkit_path
    sratoolkit_path = sra_p # update global variable
    log += f"Set SRA Toolkit path: {sra_p}\n"

```

```

fastqc_p = setup_fastqc(app_dir)
set_paths("FASTQC_PATH", fastqc_p)
log += f"Set FastQC path: {fastqc_p}\n"
fastp_p = setup_fastp(app_dir)
set_paths("FASTP_PATH", fastp_p)
log += f"Set Fastp path: {fastp_p}\n"
log += "Setup completed successfully."
return log

```

```

def sra_cmd_for_info(accession: str):
    toolkit_path = os.getenv("SRATOOLKIT_PATH") or f"{app_dir}/sratoolkit/bin"
    vdb_dump_path = os.path.join(toolkit_path, "vdb-dump")
    return f"{vdb_dump_path} {accession} --info"

```

```

def sra_cmd_to_download(accession: str,
                        alignment_filter: bool,
                        compressed: bool,
                        skip_technical: bool,
                        remove_adapter: bool,
                        spot_group: bool,
                        alignment_filter_type: str = None,
                        min_reads: int = None,
                        max_reads: int = None,
                        ar_specific: str = None,
                        ar_start: int = None,
                        ar_end: int = None,
                        member: str = None):
    toolkit_path = os.getenv("SRATOOLKIT_PATH") or f"{app_dir}/sratoolkit/bin"
    fastq_dump_path = os.path.join(toolkit_path, "fastq-dump")
    cmd_list = [fastq_dump_path, accession, '--split-3', '--outdir', f"{data_dir}/{accession}"]
    if compressed:
        cmd_list.append('--gzip')
    if alignment_filter:
        if alignment_filter_type == "split-spot":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--split-spot')
        elif alignment_filter_type == "aligned":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--aligned')
        elif alignment_filter_type == "unaligned":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--unaligned')
        elif alignment_filter_type == "aligned-region":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')

```

```

        cmd_list.append('--aligned-region')
        cmd_list.append(f'{ar_specific}:{ar_start}-{ar_end}')
    elif alignment_filter_type == "matepair-distance":
        if '--split-3' in cmd_list: cmd_list.remove('--split-3')
        cmd_list.append('--matepair-distance')
        cmd_list.append(f'{ar_start}-{ar_end}')
    if skip_technical:
        cmd_list.append('--skip-technical')
    if min_reads:
        cmd_list.extend(['--minSpotId', str(min_reads)])
    if max_reads:
        cmd_list.extend(['--maxSpotId', str(max_reads)])
    if remove_adapter:
        cmd_list.append('--clip')
    if spot_group:
        cmd_list.extend(['--spot-group', member])
    return ''.join(cmd_list)

def list_files_in_directory(directory):
    try:
        return [os.path.join(directory, f) for f in os.listdir(directory)
                if os.path.isfile(os.path.join(directory, f))]
    except Exception as e:
        return []

def compress_dir(directory, zip_filename):
    import zipfile
    with zipfile.ZipFile(zip_filename, mode='w') as zip_file:
        for root, dirs, files in os.walk(directory):
            for file in files:
                file_path = os.path.join(root, file)
                zip_file.write(file_path, arcname=os.path.relpath(file_path, directory))
    return zip_filename

def sra_download_main(accession: str,
                      alignment_filter_type: str,
                      alignment_filter: bool,
                      compressed: bool,
                      skip_technical: bool,
                      remove_adapter: bool,
                      spot_group: bool,
                      min_reads: int = None,
                      max_reads: int = None,
                      ar_specific: str = None,

```

```

        ar_start: int = None,
        ar_end: int = None,
        member: str = None):
    try:
        shutil.rmtree(data_dir)
    except FileNotFoundError:
        print("Data folder not found; proceeding.")
    except OSError as e:
        print("Error removing data folder:", e)
    sra_info_cmd = sra_cmd_for_info(accession)
    try:
        run_command_out(sra_info_cmd)
    except Exception as e:
        try:
            install_sra()
        except Exception as e:
            print("Error during SRA installation:", e)
    work_dir = os.path.join(data_dir, accession)
    os.makedirs(work_dir, mode=0o777, exist_ok=True)
    # Set the working directory environment variable for downstream steps
    os.environ["WORKING_DIR"] = work_dir
    get_data_cmd = sra_cmd_for_info(accession)
    download_cmd = sra_cmd_to_download(accession, alignment_filter, compressed,
                                       skip_technical, remove_adapter, spot_group,
                                       alignment_filter_type, min_reads, max_reads,
                                       ar_specific, ar_start, ar_end, member)
    log = f"Fetching data with: {get_data_cmd}\n"
    data_info = run_command_out(get_data_cmd)
    log += f"Data info: {data_info}\n"
    log += f"Downloading Fastq files with: {download_cmd}\n"
    download_status = run_command_out(download_cmd)
    log += f"Download status: {download_status}\n"
    zip_file = compress_dir(work_dir, os.path.join(data_dir, f"{accession}.zip"))
    print(f"Data downloaded successfully for {accession}")
    return data_info, download_status, zip_file

def fastqc_main(uploaded_files):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first.", []
    # Copy uploaded files into working directory
    for file_obj in uploaded_files:
        dest = os.path.join(working_dir, os.path.basename(file_obj.name))
        shutil.copy(file_obj.name, dest)

```

```

        print(f"Copied {file_obj.name} to {dest}") # Debug print
    output_dir = os.path.join(working_dir, 'fastqc_reports')
    os.makedirs(output_dir, exist_ok=True)
    # Process each file with FastQC
    for f in os.listdir(working_dir):
        file_path = os.path.join(working_dir, f)
        if any(ext in f for ext in [".fastq", ".fasta", ".fa", ".sam", ".bam"]):
            cmd = f"fastqc {file_path} --outdir {output_dir}"
            print(f"Running FastQC command: {cmd}") # Debug print
            try:
                # Use fastqc_path as the directory where the fastqc binary is located
                run_command_out(cmd, dir=fastqc_path)
            except Exception as e:
                print(f"Error running FastQC on {file_path}: {e}")
    # Collect reports (HTML files)
    reports = [os.path.join(output_dir, f) for f in os.listdir(output_dir)
               if f.endswith(".html")]
    if not reports:
        print("No FastQC reports were generated.")
    else:
        print("Generated FastQC reports:", reports)
    return "FastQC processing completed.", reports

def fastp_main(uploaded_files):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first.", []
    if len(uploaded_files) < 2:
        return "Please upload two FASTQ files (forward and reverse).", []
    file1, file2 = uploaded_files[0], uploaded_files[1]
    dest1 = os.path.join(working_dir, os.path.basename(file1.name))
    dest2 = os.path.join(working_dir, os.path.basename(file2.name))
    shutil.copy(file1.name, dest1)
    shutil.copy(file2.name, dest2)
    output_file_1 = os.path.join(working_dir, os.path.basename(dest1).replace('.fastq.gz',
'_trimmed.fastq.gz'))
    output_file_2 = os.path.join(working_dir, os.path.basename(dest2).replace('.fastq.gz',
'_trimmed.fastq.gz'))
    output_dir = os.path.join(working_dir, 'fastp_reports')
    os.makedirs(output_dir, exist_ok=True)
    command = (f'{fastp_path}/fastp'
               f'-i {dest1}'
               f'-o {output_file_1}'
               f'-I {dest2}')

```

```

        f" -O {output_file_2}"
        f" --detect_adapter_for_pe"
        f" -f 10 -g -l 50"
        f" -c -h {output_dir}/fastp_report.html"
        f" -w 10")
try:
    run_command_out(command)
    trimmed_dir = os.path.join(working_dir, 'trimmed')
    os.makedirs(trimmed_dir, exist_ok=True)
    shutil.move(output_file_1, os.path.join(trimmed_dir, os.path.basename(output_file_1)))
    shutil.move(output_file_2, os.path.join(trimmed_dir, os.path.basename(output_file_2)))
    trimmed_files = [os.path.join(trimmed_dir, os.path.basename(output_file_1)),
                     os.path.join(trimmed_dir, os.path.basename(output_file_2))]
    return "Fastp processing completed.", trimmed_files
except Exception as e:
    return f"Error running Fastp: {e}", []

def download_reference_genome(url=None, custom_name=None):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first."

    ref_dir = os.path.join(working_dir, "reference")
    os.makedirs(ref_dir, exist_ok=True)

    if not url:
        return "No URL provided for reference genome download."

    try:
        # Extract filename from URL
        filename = os.path.basename(url)
        if custom_name:
            # Use custom name but preserve extension
            ext = os.path.splitext(filename)[1]
            filename = f"{custom_name}{ext}"

        compressed_path = os.path.join(ref_dir, filename)

        print(f"Downloading reference genome from {url}...")
        run_command_out(f"wget -O {compressed_path} {url}")

        if filename.endswith('.gz'):
            print("Decompressing gzip file...")
            run_command_out(f"gunzip {compressed_path}")

```

```

        uncompressed_path = compressed_path[:-3] # remove .gz
elif filename.endswith('.zip'):
    print("Extracting zip file...")
    run_command_out(f'unzip {compressed_path} -d {ref_dir}')
    os.remove(compressed_path)
    uncompressed_path = None
    for f in os.listdir(ref_dir):
        if f.endswith(('fa', '.fasta', '.fna')):
            uncompressed_path = os.path.join(ref_dir, f)
            break
    if not uncompressed_path:
        return "No FASTA file found in the downloaded archive."
elif filename.endswith('.tar.gz') or filename.endswith('.tgz'):
    print("Extracting tar.gz file...")
    with tarfile.open(compressed_path, "r:gz") as tar:
        tar.extractall(path=ref_dir)
    os.remove(compressed_path)
    # Assume the first .fa/.fasta file is the reference
    uncompressed_path = None
    for f in os.listdir(ref_dir):
        if f.endswith(('fa', '.fasta', '.fna')):
            uncompressed_path = os.path.join(ref_dir, f)
            break
    if not uncompressed_path:
        return "No FASTA file found in the downloaded archive."
else:
    uncompressed_path = compressed_path

if custom_name and uncompressed_path.endswith(('fa', '.fasta', '.fna')):
    final_path = os.path.join(ref_dir,
f'{custom_name} {os.path.splitext(uncompressed_path)[1]}')
    os.rename(uncompressed_path, final_path)
    uncompressed_path = final_path

# Index the reference genome
print("Indexing reference genome...")
run_command_out(f'bwa index {uncompressed_path}')
run_command_out(f'samtools faidx {uncompressed_path}')
dict_path = uncompressed_path.replace('.fa', '.dict').replace('.fasta', '.dict').replace('.fna',
'.dict')
run_command_out(f'gatk CreateSequenceDictionary -R {uncompressed_path} -O
{dict_path}')

```



```

        return f"Reference genome downloaded and indexed successfully at
{uncompressed_path}"

    except Exception as e:
        return f"Error downloading reference genome: {str(e)}"

def load_reference_genome(ref_genome_file=None, genome_url=None,
custom_name=None):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first."

    ref_dir = os.path.join(working_dir, "reference")
    os.makedirs(ref_dir, exist_ok=True)

    if genome_url:
        # Download from URL
        return download_reference_genome(genome_url, custom_name)
    elif ref_genome_file:
        # Handle uploaded file
        dest_path = os.path.join(ref_dir, os.path.basename(ref_genome_file.name))
        if custom_name:
            # Preserve extension
            ext = os.path.splitext(ref_genome_file.name)[1]
            dest_path = os.path.join(ref_dir, f"{custom_name}{ext}")

        shutil.copy(ref_genome_file.name, dest_path)

    if dest_path.endswith(('.fa', '.fasta', '.fna')):
        try:
            # BWA index
            run_command_out(f"bwa index {dest_path}")
            # SAMtools index
            run_command_out(f"samtools faidx {dest_path}")
            # Create dictionary
            dict_path = dest_path.replace('.fa', '.dict').replace('.fasta', '.dict').replace('.fna',
'.dict')
            run_command_out(f"gatk CreateSequenceDictionary -R {dest_path} -O
{dict_path}")
            return f"Reference genome loaded and indexed successfully at {dest_path}"
        except Exception as e:
            return f"Error indexing reference genome: {e}"
        return f"Reference genome loaded successfully at {dest_path}"
    else:

```

```

        return "No reference genome file or URL provided."

def run_bwa_alignment(reads_files, ref_genome_name=None, threads=4,
algorithm="mem"):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first.", None

    # Find reference genome
    ref_dir = os.path.join(working_dir, "reference")
    if not os.path.exists(ref_dir):
        return "Reference genome directory not found. Load a reference genome first.", None

    if not ref_genome_name:
        ref_files = [f for f in os.listdir(ref_dir) if f.endswith(('.fa', '.fasta', '.fna'))]
        if not ref_files:
            return "No reference genome found in reference directory.", None
        ref_genome_name = ref_files[0]

    ref_path = os.path.join(ref_dir, ref_genome_name)

    bwa_data_dir = os.path.join(working_dir, 'bwa_index_map')
    os.makedirs(bwa_data_dir, exist_ok=True)
    ref_prefix = os.path.join(bwa_data_dir, os.path.splitext(ref_genome_name)[0] + '_ref')

    align_dir = os.path.join(working_dir, "alignment")
    os.makedirs(align_dir, exist_ok=True)

    try:
        # Index the reference genome if not already indexed
        if not os.path.exists(f'{ref_prefix}.bwt'):
            print("Indexing reference genome...")
            run_command_out(f'bwa index -a bwtsv -p {ref_prefix} {ref_path}')

        if len(reads_files) == 1:
            # Single-end reads
            reads_file = os.path.join(working_dir, os.path.basename(reads_files[0].name))
            shutil.copy(reads_files[0].name, reads_file)

            output_sam = os.path.join(align_dir, "aligned.sam")
            cmd = f'bwa {algorithm} {ref_prefix} {reads_file} -t {threads} -o {output_sam}'
        elif len(reads_files) == 2:
            reads1 = os.path.join(working_dir, os.path.basename(reads_files[0].name))
            reads2 = os.path.join(working_dir, os.path.basename(reads_files[1].name))

```

```

        shutil.copy(reads_files[0].name, reads1)
        shutil.copy(reads_files[1].name, reads2)

        output_sam = os.path.join(aligned_dir, "aligned.sam")
        cmd = f"bwa {algorithm} {ref_prefix} {reads1} {reads2} -t {threads} -o
{output_sam}"
        else:
            return "Please provide either 1 (single-end) or 2 (paired-end) reads files.", None

    print(f"Running BWA command: {cmd}")
    run_command_out(cmd)

    output_bam = output_sam.replace('.sam', '.bam')
    run_command_out(f"samtools view -S -b {output_sam} > {output_bam}")
    sorted_bam = output_bam.replace('.bam', '.sorted.bam')
    run_command_out(f"samtools sort {output_bam} -o {sorted_bam}")
    run_command_out(f"samtools index {sorted_bam}")

    os.remove(output_sam)
    os.remove(output_bam)

    return "BWA alignment completed successfully.", sorted_bam
except Exception as e:
    return f"Error during BWA alignment: {e}", None

def download_gatk_resources(analysis_dir):
    """Download required GATK resource files"""
    resources = {
        "dbSNP": {
            "vcf": "https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/Homo_sapiens_assembly38.dbsnp138.vcf",
            "idx": "https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/Homo_sapiens_assembly38.dbsnp138.vcf.idx"
        },
        "hapmap": {
            "vcf": "https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz",
            "tbi": "https://storage.googleapis.com/genomics-public-
data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz.tbi"
        }
    }

    log = ""
    for res_name, urls in resources.items():

```

```

vcf_path = os.path.join(analysis_dir, os.path.basename(urls["vcf"]))
idx_path = os.path.join(analysis_dir, os.path.basename(urls["idx"]))

if not os.path.exists(vcf_path):
    log += f"Downloading {res_name} VCF...\n"
    run_command_out(f"wget {urls['vcf']} -O {vcf_path}")
else:
    log += f"{res_name} VCF already exists.\n"

if not os.path.exists(idx_path):
    log += f"Downloading {res_name} index...\n"
    run_command_out(f"wget {urls['idx']} -O {idx_path}")
else:
    log += f"{res_name} index already exists.\n"

return log

def run_gatk_analysis(bam_file, run_mode="full", skip_downloads=False):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first.", None

    analysis_dir = os.path.join(working_dir, "gatk_analysis")
    os.makedirs(analysis_dir, exist_ok=True)

    ref_dir = os.path.join(working_dir, "reference")
    if not os.path.exists(ref_dir):
        return "Reference genome directory not found. Load a reference genome first.", None

    ref_files = [f for f in os.listdir(ref_dir) if f.endswith(('.fa', '.fasta', '.fna'))]
    if not ref_files:
        return "No reference genome found in reference directory.", None

    ref_path = os.path.join(ref_dir, ref_files[0])

    try:
        log = "Starting GATK analysis...\n"

        bam_path = os.path.join(analysis_dir, os.path.basename(bam_file.name))
        shutil.copy(bam_file.name, bam_path)
        log += f"Copied BAM file to {bam_path}\n"

        if not skip_downloads:
            log += download_gatk_resources(analysis_dir)

```

```

# Define output files
output_files = []

# 1. Add read groups
with_rg_bam = os.path.join(analysis_dir, "sample_withRG.bam")
run_command_out(f"gatk AddOrReplaceReadGroups -I {bam_path} -O {with_rg_bam}
"
                f"-ID 1 -LB lib1 -PL ILLUMINA -PU unit1 -SM sample1")
log += "Added read groups to BAM file\n"

# 2. Sort BAM file
sorted_bam = os.path.join(analysis_dir, "sample_sorted.bam")
run_command_out(f"gatk SortSam -I {with_rg_bam} -O {sorted_bam} -SO
coordinate")
log += "Sorted BAM file\n"

# 3. Mark duplicates
marked_bam = os.path.join(analysis_dir, "sample_marked.bam")
metrics_file = os.path.join(analysis_dir, "marked_dup_metrics.txt")
run_command_out(f"gatk MarkDuplicates -I {sorted_bam} -O {marked_bam} -M
{metrics_file}")
log += "Marked duplicates\n"

# 4. Create BQSR table
dbsnp_vcf = os.path.join(analysis_dir, "Homo_sapiens_assembly38.dbsnp138.vcf")
recal_table = os.path.join(analysis_dir, "sample_recal_data.table")
run_command_out(f"gatk BaseRecalibrator -I {marked_bam} -R {ref_path} "
                f"--known-sites {dbsnp_vcf} -O {recal_table}")
log += "Created BQSR table\n"

# 5. Apply BQSR
recal_bam = os.path.join(analysis_dir, "sample_recal.bam")
run_command_out(f"gatk ApplyBQSR -R {ref_path} -I {marked_bam} "
                f"--bqsr-recal-file {recal_table} -O {recal_bam}")
log += "Applied base quality score recalibration\n"

# 6. Variant calling based on run mode
if run_mode in ["full", "germline"]:
    # Germline variant calling
    vcf_output = os.path.join(analysis_dir, "sample_variants.vcf.gz")
    run_command_out(f"gatk HaplotypeCaller -R {ref_path} -I {recal_bam} "
                    f"-O {vcf_output} -ERC GVCF")
    output_files.append(vcf_output)

```

```

log += "Performed germline variant calling\n"

if run_mode == "full":
    # Additional filtering
    filtered_vcf = os.path.join(analysis_dir, "sample_filtered.vcf.gz")
    run_command_out(f'gatk VariantFiltration -R {ref_path} -V {vcf_output} "
        f'-O {filtered_vcf} --filter-expression 'QD < 2.0 || FS > 60.0 || MQ < 40.0'
"
        f'--filter-name 'basic_filter"'')
    output_files.append(filtered_vcf)
    log += "Applied variant filtering\n"

if run_mode in ["full", "somatic"]:
    # Somatic variant calling
    somatic_vcf = os.path.join(analysis_dir, "sample_somatic.vcf.gz")
    run_command_out(f'gatk Mutect2 -R {ref_path} -I {recal_bam} "
        f'-O {somatic_vcf} --germline-resource {dbsnp_vcf}')
    output_files.append(somatic_vcf)
    log += "Performed somatic variant calling\n"

if run_mode == "full":
    # Filter somatic variants
    filtered_somatic = os.path.join(analysis_dir, "sample_somatic_filtered.vcf.gz")
    run_command_out(f'gatk FilterMutectCalls -R {ref_path} -V {somatic_vcf} "
        f'-O {filtered_somatic}')
    output_files.append(filtered_somatic)
    log += "Filtered somatic variants\n"

for vcf in output_files:
    if vcf.endswith(".vcf.gz"):
        run_command_out(f'gatk IndexFeatureFile -I {vcf}')

# Return the primary output file
if output_files:
    primary_output = output_files[0]
    if run_mode == "full":
        primary_output = filtered_vcf if "filtered_vcf" in locals() else output_files[-1]
    return log, primary_output
else:
    return log + "Analysis completed but no variant files were generated.", None

except Exception as e:
    return f'Error during GATK analysis: {str(e)}', None

```

```

def enable_spot_group(esg):
    return gr.update(visible=True) if esg else gr.update(visible=False)

def enable_reads_count(erc):
    return (gr.update(visible=True), gr.update(visible=True)) if erc else
    (gr.update(visible=False), gr.update(visible=False))

def enable_alignment_filter(eaf):
    return gr.update(visible=True) if eaf else gr.update(visible=False)

def enable_alignment_filter_type(eaft):
    if eaft == "aligned-region":
        return gr.update(visible=True), gr.update(visible=True), gr.update(visible=True)
    elif eaft == "matepair-distance":
        return gr.update(visible=False), gr.update(visible=True), gr.update(visible=True)
    else:
        return gr.update(visible=False), gr.update(visible=False), gr.update(visible=False)

def get_fastq(accession, alignment_filter_type, alignment_filter, compression,
              skip_technical, remove_adapter, spot_group, min_reads, max_reads,
              ar_specific, ar_start, ar_end, member):
    print("Received parameters:")
    print("accession:", accession)
    print("alignment_filter_type:", alignment_filter_type)
    print("alignment_filter:", alignment_filter)
    print("compression:", compression)
    print("skip_technical:", skip_technical)
    print("remove_adapter:", remove_adapter)
    print("spot_group:", spot_group)
    print("min_reads:", min_reads)
    print("max_reads:", max_reads)
    print("ar_specific:", ar_specific)
    print("ar_start:", ar_start)
    print("ar_end:", ar_end)
    print("member:", member)
    data_info, download_status, file_zip = sra_download_main(accession,
alignment_filter_type, alignment_filter,
                                                              compression, skip_technical, remove_adapter,
                                                              spot_group, min_reads, max_reads,
                                                              ar_specific, ar_start, ar_end, member)
    return data_info, download_status, file_zip

with gr.Blocks(title="NGS-Precision Medicine Toolkit") as demo:
    gr.Markdown("# NGS-Precision Medicine Toolkit")

```

```

gr.Markdown("Streamlining NGS workflows for Applications of Precision Medicine")

with gr.Tab("Setup Environment"):
    gr.Markdown("## Setup Environment & Install Dependencies")
    sudo_input = gr.Textbox(label="Enter Sudo Password", type="password",
placeholder="Sudo Password")
    setup_button = gr.Button("Run Setup")
    setup_output = gr.Textbox(label="Setup Log", interactive=False)
    setup_button.click(fn=run_setup, inputs=sudo_input, outputs=setup_output)

with gr.Tab("SRA Downloader") as webui:
    gr.Markdown("## SRA Toolkit")
    with gr.Row():
        with gr.Column():
            accession = gr.Textbox(label="Enter SRA Number", placeholder="ERR11468775",
interactive=True)
            with gr.Column():
                compression = gr.Checkbox(label="Compress", value=True)
                skip_technical = gr.Checkbox(label="Skip Technical details", value=False)
                remove_adapter = gr.Checkbox(label="Remove Adapter", value=False)
                spot_group = gr.Checkbox(label="Enable Spot Grouping", value=False)
                member = gr.Textbox(label="Member", placeholder="Enter Member",
interactive=True, visible=False)
            with gr.Column():
                apply_min_max_reads = gr.Checkbox(label="Enable Min & Max Reads Count",
value=False)
                min_reads = gr.Number(label="Minimum Reads", visible=False)
                max_reads = gr.Number(label="Maximum Reads", visible=False)
            with gr.Column():
                alignment_filter = gr.Checkbox(label="Apply Alignment Filters", value=False)
                alignment_filter_type = gr.Radio(label="Alignment Filter Type",
choices=["split-spot", "aligned", "unaligned", "aligned-region",
"matepair-distance"],
visible=False)

        with gr.Column():
            ar_specific = gr.Textbox(label="Aligned Region Specific", placeholder="Enter
Specific Region", interactive=True, visible=False)
            ar_start = gr.Number(label="Aligned Region Start", visible=False)
            ar_end = gr.Number(label="Aligned Region End", visible=False)
            apply_min_max_reads.change(fn=enable_reads_count,
inputs=apply_min_max_reads, outputs=[min_reads, max_reads])
            alignment_filter.change(fn=enable_alignment_filter, inputs=alignment_filter,
outputs=[alignment_filter_type])

```



```

        alignment_filter_type.change(fn=enable_alignment_filter_type,
inputs=alignment_filter_type, outputs=[ar_specific, ar_start, ar_end])
        spot_group.change(fn=enable_spot_group, inputs=spot_group, outputs=[member])
        btn2 = gr.Button(value="Download SRA Data")
        with gr.Row():
            data = gr.Textbox(label="Data", placeholder="Data", lines=10, interactive=True)
            download_status = gr.Textbox(label="Download Status", placeholder="Download
Status", lines=5, interactive=True)
            files = gr.File(label="Download your File")
        btn2.click(fn=get_fastq,
            inputs=[accession, alignment_filter_type, alignment_filter, compression,
skip_technical,
                remove_adapter, spot_group, min_reads, max_reads, ar_specific, ar_start,
ar_end, member],
            outputs=[data, download_status, files])

        with gr.Tab("FastQC"):
            gr.Markdown("## FastQC Tool")
            gr.Markdown("Upload unzipped FASTQ/FASTA files (forward and reverse reads).")
            fastqc_files_input = gr.File(label="Upload FASTQ/FASTA Files",
file_count="multiple")
            fastqc_run_button = gr.Button("Run FastQC")
            fastqc_log = gr.Textbox(label="FastQC Log", interactive=False)
            fastqc_reports = gr.File(label="FastQC Reports", file_count="multiple")
            fastqc_run_button.click(fn=fastqc_main,
                inputs=fastqc_files_input,
                outputs=[fastqc_log, fastqc_reports])

        with gr.Tab("Fastp"):
            gr.Markdown("## Fastp Tool")
            gr.Markdown("Upload FASTQ files for adapter trimming (Forward and Reverse).")
            fastp_files_input = gr.File(label="Upload FASTQ Files", file_count="multiple")
            fastp_run_button = gr.Button("Run Fastp")
            fastp_log = gr.Textbox(label="Fastp Log", interactive=False)
            fastp_reports = gr.File(label="Trimmed FASTQ Files", file_count="multiple")
            fastp_run_button.click(fn=fastp_main,
                inputs=fastp_files_input,
                outputs=[fastp_log, fastp_reports])

        with gr.Tab("Reference Genome"):
            gr.Markdown("## Reference Genome Selection")
            with gr.Row():
                with gr.Column():
                    gr.Markdown("#### Option 1: Upload Local File")

```

```

ref_genome_file = gr.File(label="Upload Reference Genome (FASTA/FASTQ)")
custom_name_upload = gr.Textbox(label="Custom Name (optional)",
placeholder="e.g., hg38_chr18")
with gr.Column():
    gr.Markdown("### Option 2: Download from URL")
    genome_url = gr.Textbox(label="Reference Genome URL",
placeholder="e.g.,
https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr18.fa.gz")
    custom_name_url = gr.Textbox(label="Custom Name (optional)",
placeholder="e.g., hg38_chr18")
    example_urls = gr.Dropdown(
        label="Example URLs",
        choices=[
            "UCSC hg38 chr18",
            "UCSC hg38 chrM",
            "GRCh38 primary assembly"
        ],
        value="Select an example"
    )

    def update_url(example):
        urls = {
            "UCSC hg38 chr18":
"https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr18.fa.gz",
            "UCSC hg38 chrM":
"https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chrM.fa.gz",
            "GRCh38 primary assembly":
"https://ftp.ncbi.nlm.nih.gov/genomes/all/GCA/000/001/405/GCA_000001405.15_GRCh38/s
eqs_for_alignment_pipelines.ucsc_ids/GCA_000001405.15_GRCh38_no_alt_analysis_set.f
a.gz"
        }
        return gr.update(value=urls.get(example, ""))

    example_urls.change(fn=update_url, inputs=example_urls, outputs=genome_url)

ref_genome_button = gr.Button("Load Reference Genome")
ref_genome_log = gr.Textbox(label="Reference Genome Log", interactive=False)

ref_genome_button.click(
    fn=load_reference_genome,
    inputs=[ref_genome_file, genome_url],
    outputs=ref_genome_log
)

```

```

with gr.Tab("BWA Alignment"):
    gr.Markdown("## BWA Alignment")
    with gr.Row():
        with gr.Column():
            reads_file = gr.File(label="Upload Reads File (FASTQ)", file_count="multiple")
            ref_genome_dropdown = gr.Dropdown(
                label="Select Reference Genome",
                choices=[],
                interactive=True
            )

            def update_ref_genome_dropdown():
                working_dir = os.getenv("WORKING_DIR")
                if not working_dir:
                    return gr.update(choices=[])

                ref_dir = os.path.join(working_dir, "reference")
                if not os.path.exists(ref_dir):
                    return gr.update(choices=[])

                ref_files = [f for f in os.listdir(ref_dir) if f.endswith(('fa', 'fasta', 'fna'))]
                return gr.update(choices=ref_files)

        with gr.Column():
            algorithm = gr.Radio(
                label="Alignment Algorithm",
                choices=["mem", "aln", "bwasw"],
                value="mem",
                interactive=True
            )
            threads = gr.Slider(
                label="Number of Threads",
                minimum=1,
                maximum=32,
                value=4,
                step=1,
                interactive=True
            )

    bwa_run_button = gr.Button("Run BWA Alignment")
    bwa_log = gr.Textbox(label="BWA Log", interactive=False)
    aligned_output = gr.File(label="Aligned Output (BAM)")

    # Update reference genome dropdown when tab is selected

```

```

demo.load(
    fn=update_ref_genome_dropdown,
    inputs=[],
    outputs=[ref_genome_dropdown],
    queue=False
)

bwa_run_button.click(
    fn=run_bwa_alignment,
    inputs=[reads_file, ref_genome_dropdown, threads, algorithm],
    outputs=[bwa_log, aligned_output]
)

with gr.Tab("GATK Analysis"):
    gr.Markdown("## GATK Variant Analysis")
    with gr.Row():
        with gr.Column():
            bam_file = gr.File(label="Upload Aligned Reads (BAM)", file_count="single")
            ref_genome_dropdown = gr.Dropdown(
                label="Select Reference Genome",
                choices=[],
                interactive=True
            )

            def update_ref_genome_dropdown():
                working_dir = os.getenv("WORKING_DIR")
                if not working_dir:
                    return gr.update(choices=[])

                ref_dir = os.path.join(working_dir, "reference")
                if not os.path.exists(ref_dir):
                    return gr.update(choices=[])

                ref_files = [f for f in os.listdir(ref_dir) if f.endswith(('fa', 'fasta', 'fna'))]
                return gr.update(choices=ref_files)

            ref_genome_dropdown.change(update_ref_genome_dropdown)

        with gr.Column():
            analysis_mode = gr.Radio(
                label="Analysis Mode",
                choices=["Full", "Germline", "Somatic"],
                value="Full",
                interactive=True
            )

            skip_downloads = gr.Checkbox(

```

```

        label="Skip downloading resource files (use existing)",
        value=False,
        interactive=True
    )

    gatk_run_button = gr.Button("Run GATK Analysis")
    gatk_log = gr.Textbox(label="GATK Log", interactive=False, lines=10)
    vcf_output = gr.File(label="Variant Call File (VCF)")

    # Update reference genome dropdown when tab is selected
    demo.load(
        fn=update_ref_genome_dropdown,
        inputs=[],
        outputs=[ref_genome_dropdown],
        queue=False
    )

    gatk_run_button.click(
        fn=run_gatk_analysis,
        inputs=[bam_file, analysis_mode, skip_downloads],
        outputs=[gatk_log, vcf_output]
    )

    try:
        webui.queue(default_concurrency_limit=25)
    except Exception as e:
        print(f"Queue error: {e}")

demo.launch()

```

CHAPTER 7: RESULTS

RESULTS

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

[Setup Environment](#) [SRA Downloader](#) [FastQC](#) [Fastp](#) [Reference Genome](#) [BWA Alignment](#) [GATK Analysis](#)

Setup Environment & Install Dependencies

Enter Sudo Password

Sudo Password

Run Setup

Setup Log

Use via API  · Built with Gradio  · Settings 

Figure 8: Homepage of ‘Setup Environment’ tab

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

[Setup Environment](#) [SRA Downloader](#) [FastQC](#) [Fastp](#) [Reference Genome](#) [BWA Alignment](#) [GATK Analysis](#)

Setup Environment & Install Dependencies

Enter Sudo Password

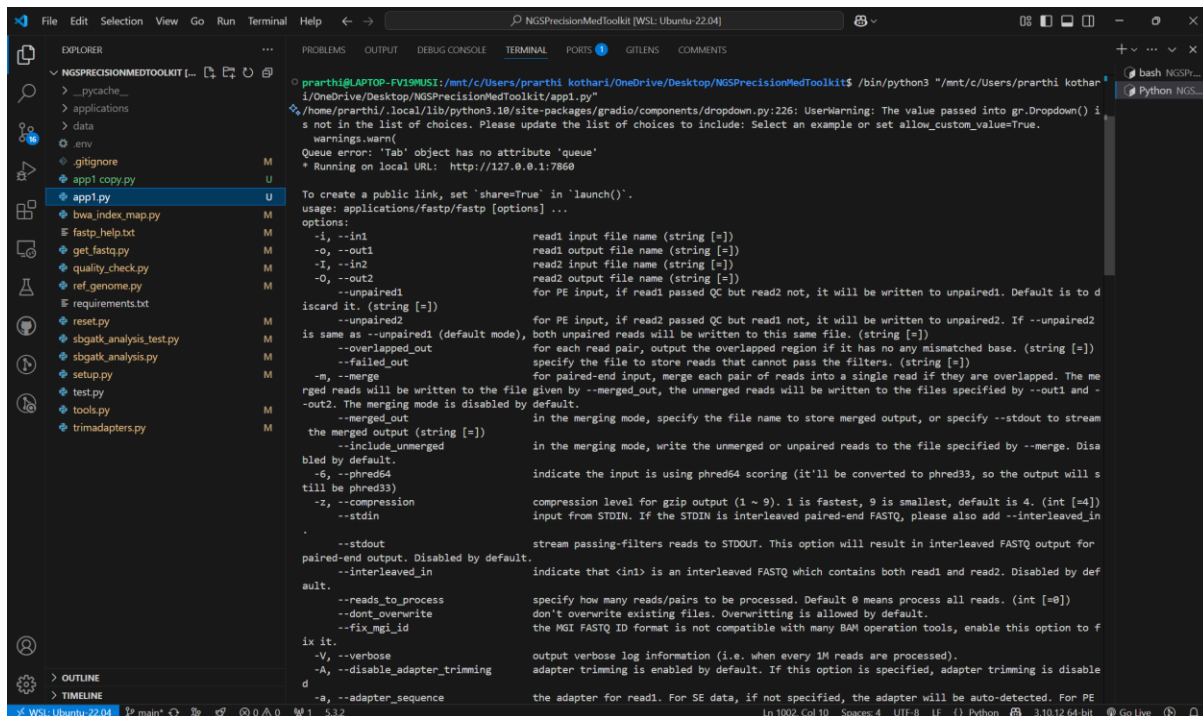
Run Setup

Setup Log

```
Get:14 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [2210 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [403 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [3241 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [574 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1198 kB]
Get:19 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [340 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [295 kB]
Get:21 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [3122 kB]
Get:22 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [554 kB]
Get:23 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [970 kB]
Get:24 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [208 kB]
Fetched 16.3 MB in 5s (3198 kB/s)
Reading package lists...
Error: [sudo] password for prarthi:

Dependencies installed with sudo.
Set SRA Toolkit path: applications/sratoolkit/bin
Set FastQC path: applications/fastqc
Set Fastp path: applications/fastp
Setup completed successfully.
```

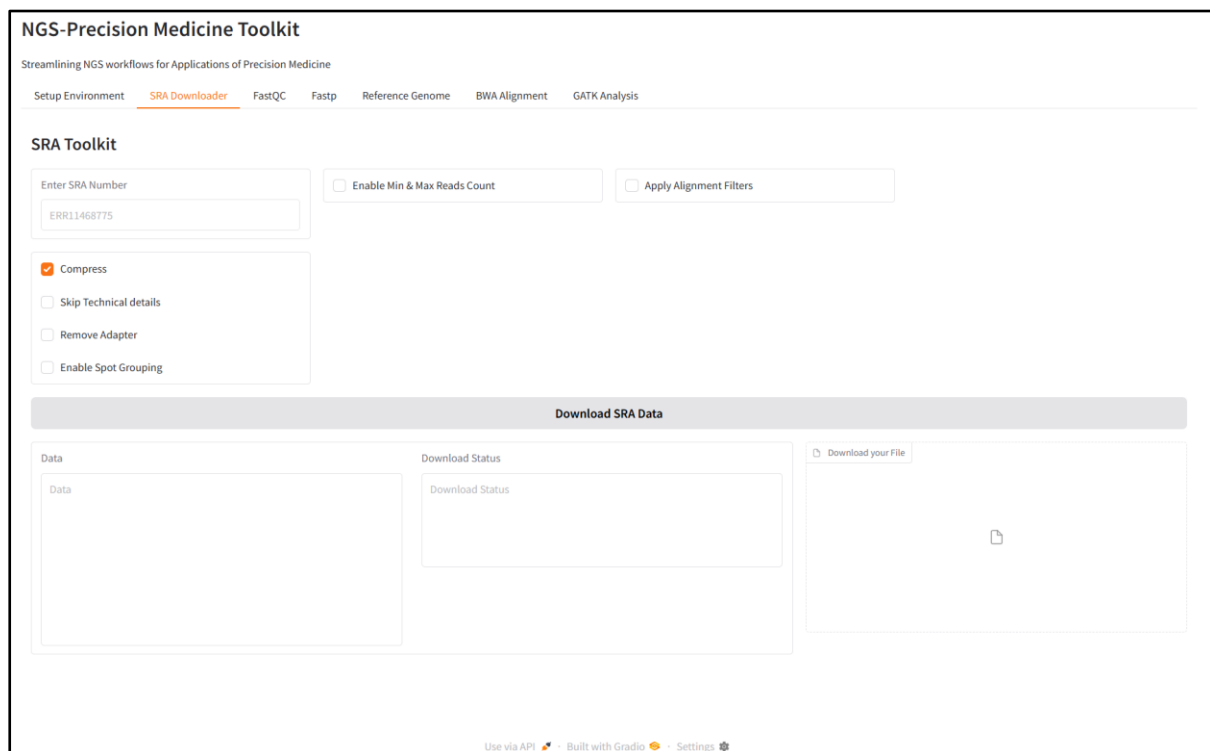
Figure 9: Entering the password and installing dependencies



```
prarthi@LAPTOP-FV19MUSI:/mnt/c/Users/prarthi.kothari/OneDrive/Desktop/NGSPrecisionMedToolkit$ /bin/python3 "/mnt/c/Users/prarthi.kothari/OneDrive/Desktop/NGSPrecisionMedToolkit/app1.py"
/home/prarthi/.local/lib/python3.10/site-packages/gradio/components/dropdown.py:226: UserWarning: The value passed into gr.Dropdown() is not in the list of choices. Please update the list of choices to include: Select an example or set allow_custom_value=True.
  warnings.warn(
Queue error: 'Tab' object has no attribute 'queue'
* Running on local URL: http://127.0.0.1:7860

To create a public link, set 'share=True' in 'launch()'.
usage: applications/fastp/fastp [options] ...
options:
  -i, --in1          read1 input file name (string [=])
  -o, --out1         read1 output file name (string [=])
  -I, --in2          read2 input file name (string [=])
  -O, --out2         read2 output file name (string [=])
  --unpaired1        for PE input, if read1 passed QC but read2 not, it will be written to unpaired1. Default is to discard it. (string [=])
  --unpaired2        for PE input, if read2 passed QC but read1 not, it will be written to unpaired2. If --unpaired2 is same as --unpaired1 (default mode), both unpaired reads will be written to this same file. (string [=])
  --overlapped_out   for each read pair, output the overlapped region if it has no any mismatched base. (string [=])
  --failed_out       specify the file to store reads that cannot pass the filters. (string [=])
  -B, --merge        for paired-end input, merge each pair of reads into a single read if they are overlapped. The merged reads will be written to the file given by --merged_out, the unmerged reads will be written to the files specified by --out1 and --out2. The merging mode is disabled by default.
  --merged_out       in the merging mode, specify the file name to store merged output, or specify --stdout to stream the merged output (string [=])
  --include_unmerged in the merging mode, write the unmerged or unpaired reads to the file specified by --merge. Disabled by default.
  -S, --phred64      indicate the input is using phred64 scoring (it'll be converted to phred33, so the output will still be phred33)
  -Z, --compression compression level for gzip output (1 ~ 9). 1 is fastest, 9 is smallest, default is 4. (int [=4])
  --stdin            input from STDIN. If the STDIN is interleaved paired-end FASTQ, please also add --interleaved_in
  --stdout           stream passing-filters reads to STDOUT. This option will result in interleaved FASTQ output for paired-end output. Disabled by default.
  --interleaved_in   indicate that <in1> is an interleaved FASTQ which contains both read1 and read2. Disabled by default.
  --reads_to_process specify how many reads/pairs to be processed. Default @ means process all reads. (int [=0])
  --dont_overwrite   don't overwrite existing files. Overwriting is allowed by default.
  --fix_mgi_id       the MGI FASTQ ID format is not compatible with many BAM operation tools, enable this option to fix it.
  -V, --verbose      output verbose log information (i.e. when every 1M reads are processed).
  -A, --disable_adapter_trimming adapter trimming is enabled by default. If this option is specified, adapter trimming is disabled.
  -D, --adapter_sequence the adapter for read1. For SE data, if not specified, the adapter will be auto-detected. For PE
```

Figure 10: Backend code running for setting up the environment and installing the dependencies after entering the sudo password



NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment **SRA Downloader** FastQC Fastp Reference Genome BWA Alignment GATK Analysis

SRA Toolkit

Enter SRA Number
ERR11468775

☐ Enable Min & Max Reads Count ☐ Apply Alignment Filters

☒ Compress
☐ Skip Technical details
☐ Remove Adapter
☐ Enable Spot Grouping

Download SRA Data

Data
Data

Download Status
Download Status

Download your File

Use via API Built with Gradio Settings

Figure 11: Homepage of ‘SRA Downloader’ tab

NGS-Precision Medicine Toolkit
Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment **SRA Downloader** FastQC Fastp Reference Genome BWA Alignment GATK Analysis

SRA Toolkit

Enter SRA Number
ERR11468775

☒ Enable Min & Max Reads Count
Minimum Reads
0
Maximum Reads
100

☒ Apply Alignment Filters
Alignment Filter Type
☐ split-spot ☐ aligned ☐ unaligned
☐ aligned-region ☐ matepair-distance

☒ Compress
☐ Skip Technical details
☐ Remove Adapter
☐ Enable Spot Grouping

Download SRA Data

Data
acc : ERR11468775
path : https://sra-pub-run-
odp.s3.amazonaws.com/sra/ERR11468775/ERR11468775
size : 49,663,706,774
type : Database
platf : SRA_PLATFORM_ILLUMINA
SEQ : 410,126,452
SCHEMA : NCBI:SRA:illumina:db#2
TIME : 0x0000000065de426e (02/28/2024 01:43)
FMT : sharq
FMTVER : 3.0.11
LDR : general-loader.3.0.8
LDRVER : 3.0.8
LDRDATE : Sep 11 2023 (9/11/2023 0:0)

Download Status
Read 100 spots for ERR11468775
Written 100 spots for ERR11468775

Download your File
ERR11468775.zip 21.7 KB

Figure 12: Downloading the FASTA reads for the required SRA ID along with various filters of choice

```

--overlap_diff_percent_limit the maximum percentage of mismatched bases to detect overlapped region of PE reads. This will af
fect overlap analysis based PE merge, adapter trimming and correction. Default 20 means 20%. (int [=20])
-U, --umi enable unique molecular identifier (UMI) preprocessing
    ne (string [=]) specify the location of UMI, can be {index1/index2/read1/read2/per_index/per_read, default is no
    --umi_len if the UMI is in read1/read2, its length should be provided (int [=0])
    --umi_prefix if specified, an underline will be used to connect prefix and UMI (i.e. prefix=UMI, UMI=AATTCG,
final=UMI_AATTCG). No prefix by default (string [=])
    --umi_skip if the UMI is in read1/read2, fastp can skip several bases following UMI, default is 0 (int [=0])
)
--umi_delim delimiter to use between the read name and the UMI, default is : (string [=:])
-P, --overrepresentation_analysis enable overrepresented sequence analysis.
-P, --overrepresentation_sampling one in (--overrepresentation_sampling) reads will be computed for overrepresentation analysis (1
~10000), smaller is slower, default is 20. (int [=20])
-j, --json the json format report file name (string [=fastp.json])
-h, --html the html format report file name (string [=fastp.html])
-R, --report_title the report title (string [=fastp report])
-w, --thread worker thread number, default is 3 (int [=3])
-s, --split split output by limiting total split file number with this option (2-999), a sequential number p
refix will be added to output name ( @001.out.fq, @002.out.fq...), disabled by default (int [=0])
-S, --split_by_lines split output by limiting lines of each file with this option(>=1000), a sequential number prefix
will be added to output name ( @001.out.fq, @002.out.fq...), disabled by default (long [=0])
-d, --split_prefix_digits the digits for the sequential number padding (1-10), default is 4, so the filename will be padde
d as @001.xxx, 0 to disable padding (int [=4])
--cut_by_quality5 DEPRECATED, use --cut_front instead.
--cut_by_quality3 DEPRECATED, use --cut_tail instead.
--cut_by_quality_aggressive DEPRECATED, use --cut_right instead.
--discard_unmerged DEPRECATED, no effect now, see the introduction for merging.
-h, --help print this message

Received parameters:
accession: ERR11468775
alignment_filter_type: None
compression: True
skip_technical: False
remove_adapter: False
spot_group: False
min_reads: 0
max_reads: 100
ar_specific:
ar_start: 0
ar_end: 0
member:
Data downloaded successfully for ERR11468775

```

Figure 13: Backend code running for downloading the FASTA reads using the SRA Downloader

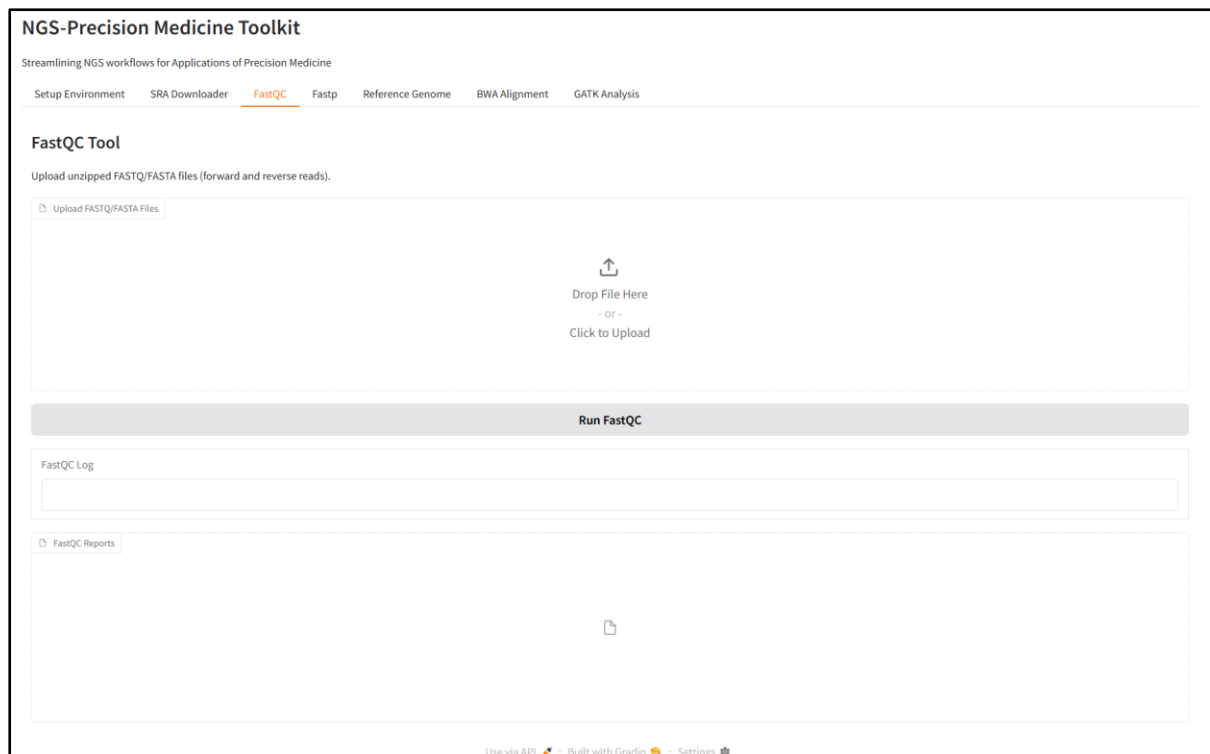


Figure 14: Homepage of ‘FastQC Tool’ tab

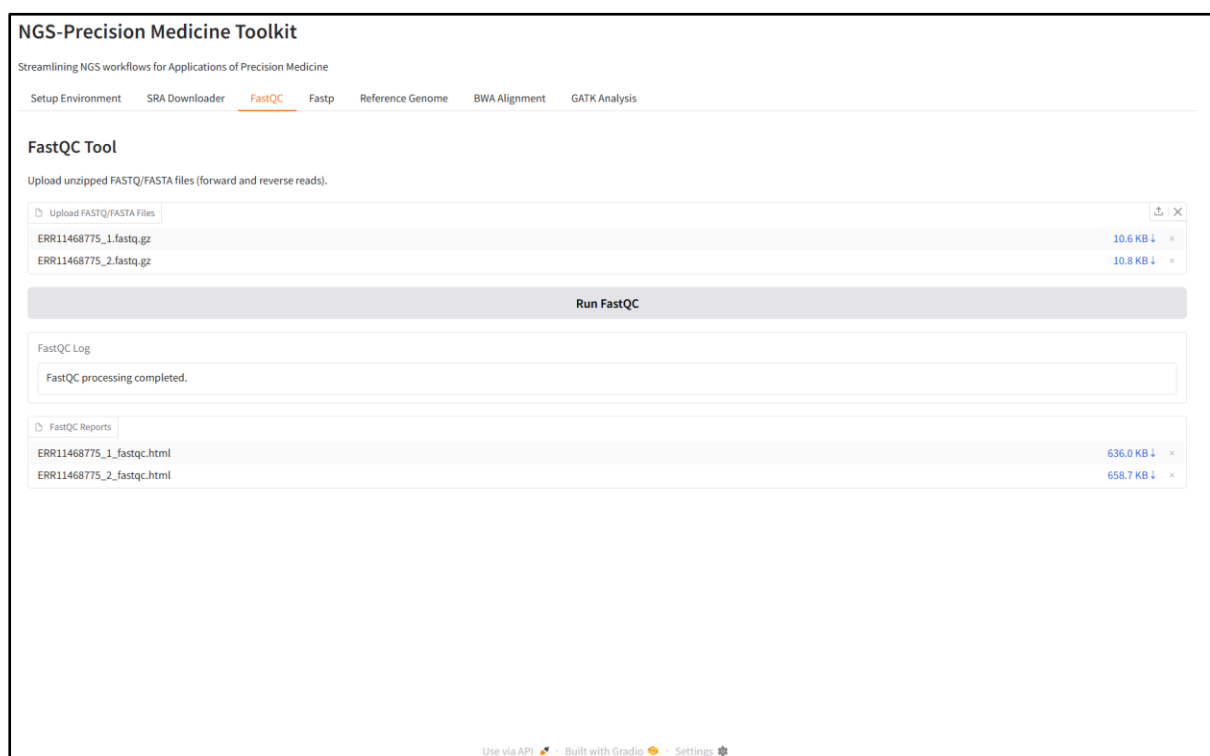


Figure 15: Analyzing the FASTQ reads and generating the quality check reports using the FastQC Tool

```

--umi_delim          delimiter to use between the read name and the UMI, default is : (string [=])
-p, --overrepresentation_analysis  enable overrepresented sequence analysis.
-P, --overrepresentation_sampling  one in (--overrepresentation_sampling) reads will be computed for overrepresentation analysis (1,
~10000), smaller is slower, default is 20. (int [=20])
-j, --json            the json format report file name (string [=fastp.json])
-h, --html            the html format report file name (string [=fastp.html])
-r, --report_title     should be quoted with ' or ", default is "fastp report" (string [=fastp report])
-w, --thread          worker thread number, default is 3 (int [=3])
-s, --split           split output by limiting total split file number with this option (2-999), a sequential number p
refix will be added to output name ( 0001.out.fq, 0002.out.fq...), disabled by default (int [=0])
-S, --split_by_lines  split output by limiting lines of each file with this option(>=1000), a sequential number prefix
will be added to output name ( 0001.out.fq, 0002.out.fq...), disabled by default (long [=0])
-d, --split_prefix_digits  the digits for the sequential number padding (1-10), default is 4, so the filename will be padde
d as 0001.xxx, 0 to disable padding (int [=4])
--cut_by_quality5     DEPRECATED, use --cut_front instead.
--cut_by_quality3     DEPRECATED, use --cut_tail instead.
--cut_by_quality_aggressive  DEPRECATED, use --cut_right instead.
--discard_unmerged    DEPRECATED, no effect now, see the introduction for merging.
->, --help            print this message

Received parameters:
accession: ERR11468775
alignment_filter_type: None
alignment_filter: True
compression: True
skip_technical: False
remove_adapter: False
spot_group: False
min_reads: 0
max_reads: 100
ar_specific:
ar_start: 0
ar_end: 0
member:
Data downloaded successfully for ERR11468775
Copied /tmp/gradio/887ace6125aa4fb451954b828f885cb514e99f027cef89108458c751473a8f83/ERR11468775_1.fastq.gz to data/ERR11468775/ERR11468
775_1.fastq.gz
Copied /tmp/gradio/824fa753d7f492968ded75d093b1a0483b247d4b3c352d75695912a06b1718be/ERR11468775_2.fastq.gz to data/ERR11468775/ERR11468
775_2.fastq.gz
Running FastQC command: fastqc data/ERR11468775/ERR11468775_1.fastq.gz --outdir data/ERR11468775/fastqc_reports
Started analysis of ERR11468775_1.fastq.gz
Running FastQC command: fastqc data/ERR11468775/ERR11468775_2.fastq.gz --outdir data/ERR11468775/fastqc_reports
Started analysis of ERR11468775_2.fastq.gz
Generated FastQC reports: ['data/ERR11468775/fastqc_reports/ERR11468775_1_fastqc.html', 'data/ERR11468775/fastqc_reports/ERR11468775_2
_fastqc.html']

```

Figure 16: Backend code running for analyzing the FASTQ reads and generating the quality check reports using the FastQC Tool

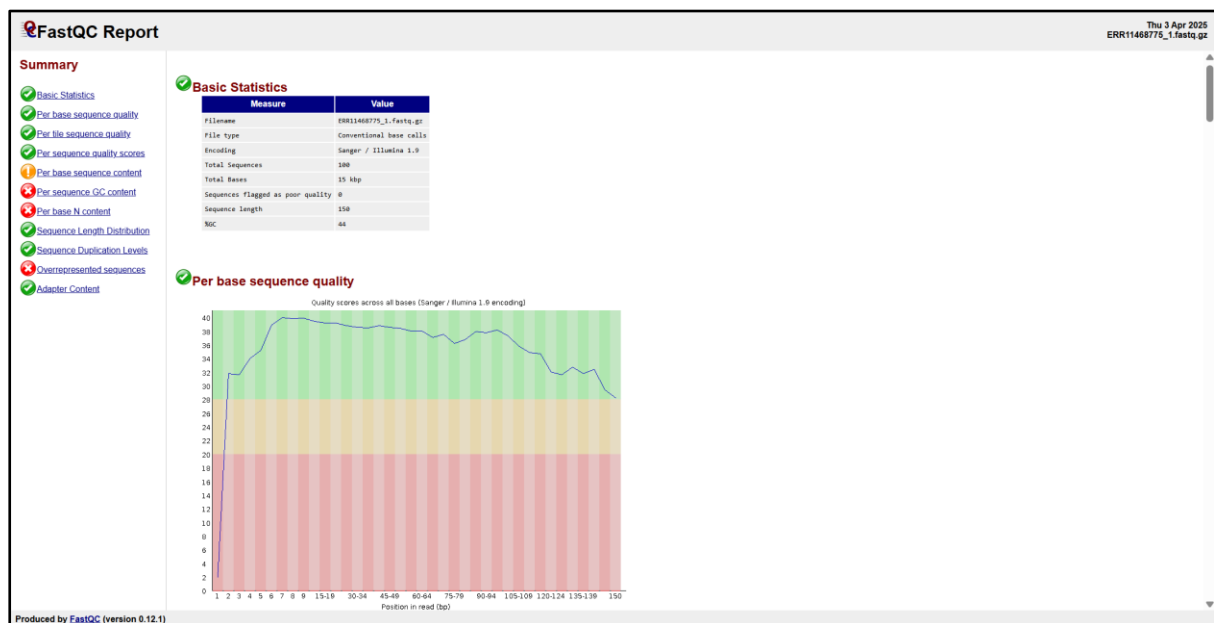


Figure 17: FASTQC Report for forward read ‘ERR11468775_1.fastq.gz’

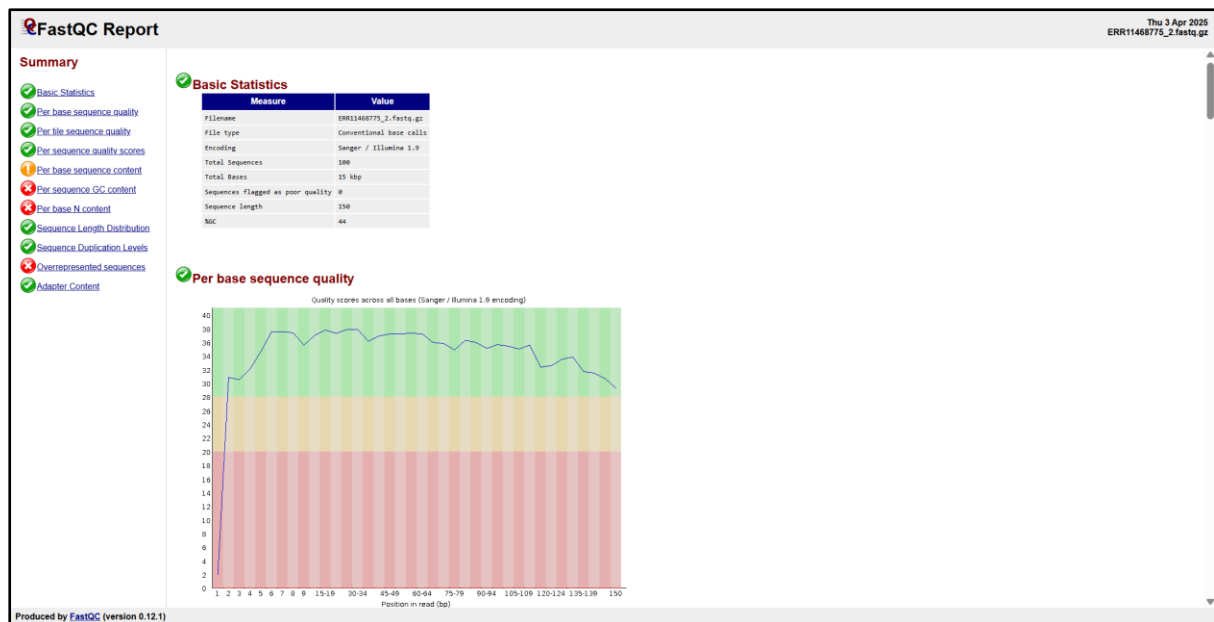


Figure 18: FASTQC Report for reverse read ‘ERR11468775_2.fastq.gz’

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment SRA Downloader FastQC **Fastp** Reference Genome BWA Alignment GATK Analysis

Fastp Tool

Upload FASTQ files for adapter trimming (Forward and Reverse).

Upload FASTQ Files

Drop File Here
- or -
Click to Upload

Run Fastp

Fastp Log

Trimmed FASTQ Files

Use via API Built with Gradle Settings

Figure 19: Homepage for ‘Fastp’ Tab

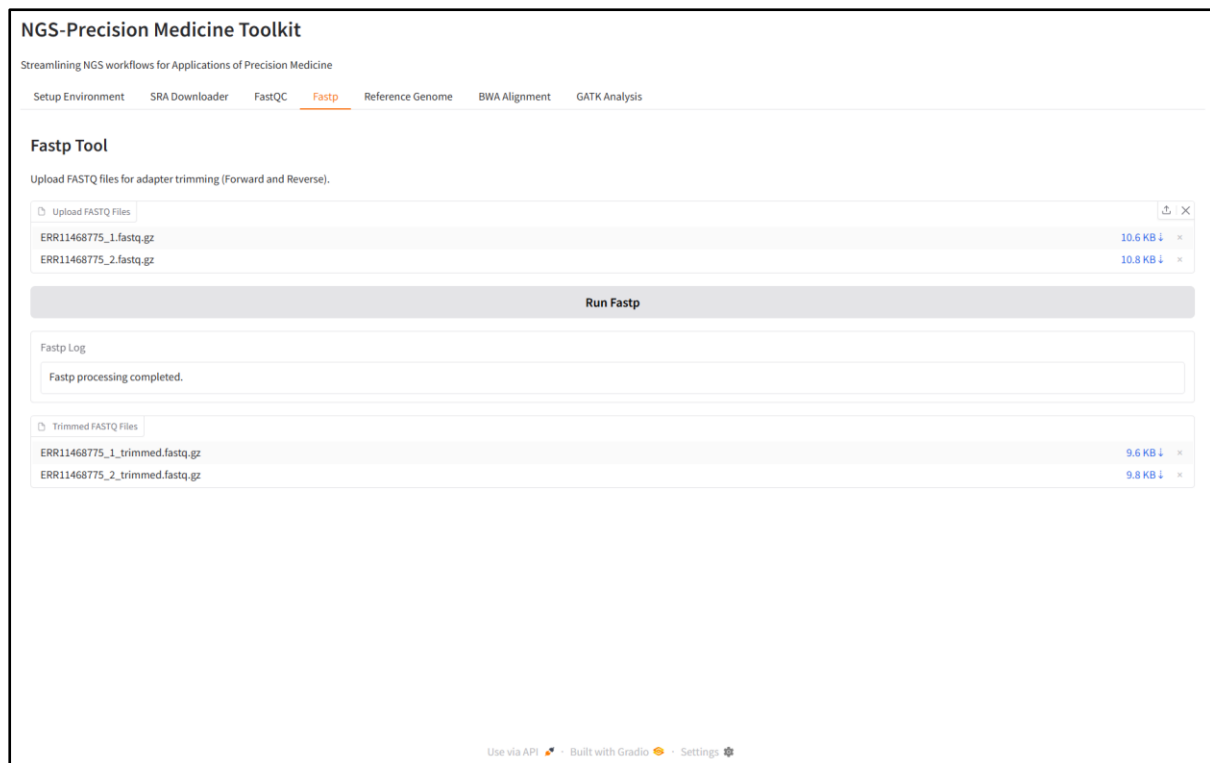


Figure 20: Analyzing the FASTQ reads and trimming the adapters to generate the processed files using Fastp Tool

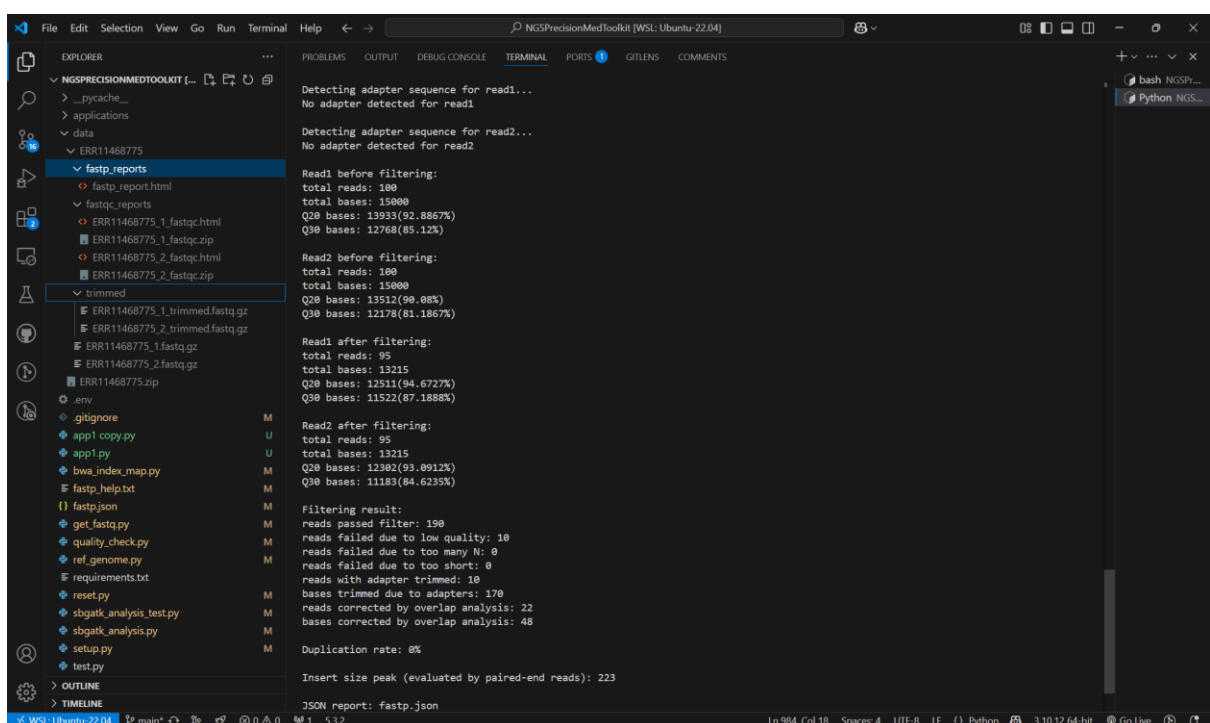


Figure 21: Backend code running for analyzing the FASTQ reads and trimming the adapters to generate the processed files using Fastp tool

fastp report

Summary

General

fastp version:	0.24.0 (https://github.com/openGene/fastp)
sequencing:	paired end (150 cycles + 150 cycles)
mean length before filtering:	150bp, 150bp
mean length after filtering:	139bp, 139bp
duplication rate:	0.000000%
Insert size peak:	223

Before filtering

total reads:	200
total bases:	30.000000 K
Q20 bases:	27.445000 K (91.483333%)
Q30 bases:	24.946000 K (83.153333%)
GC content:	43.756667%

After filtering

total reads:	190
total bases:	26.430000 K
Q20 bases:	24.813000 K (93.881952%)
Q30 bases:	22.705000 K (85.906167%)
GC content:	43.847900%

Filtering result

reads passed filters:	190 (95.000000%)
reads corrected:	22 (11.000000%)
bases corrected:	48 (0.160000%)
reads with low quality:	10 (5.000000%)
reads with too many N:	0 (0.000000%)

Figure 22: fastp report for 'ERR11468775' after filtering and trimming the adapters using fastp tool

Before filtering: read1: base contents

Value of each position will be shown on mouse over.

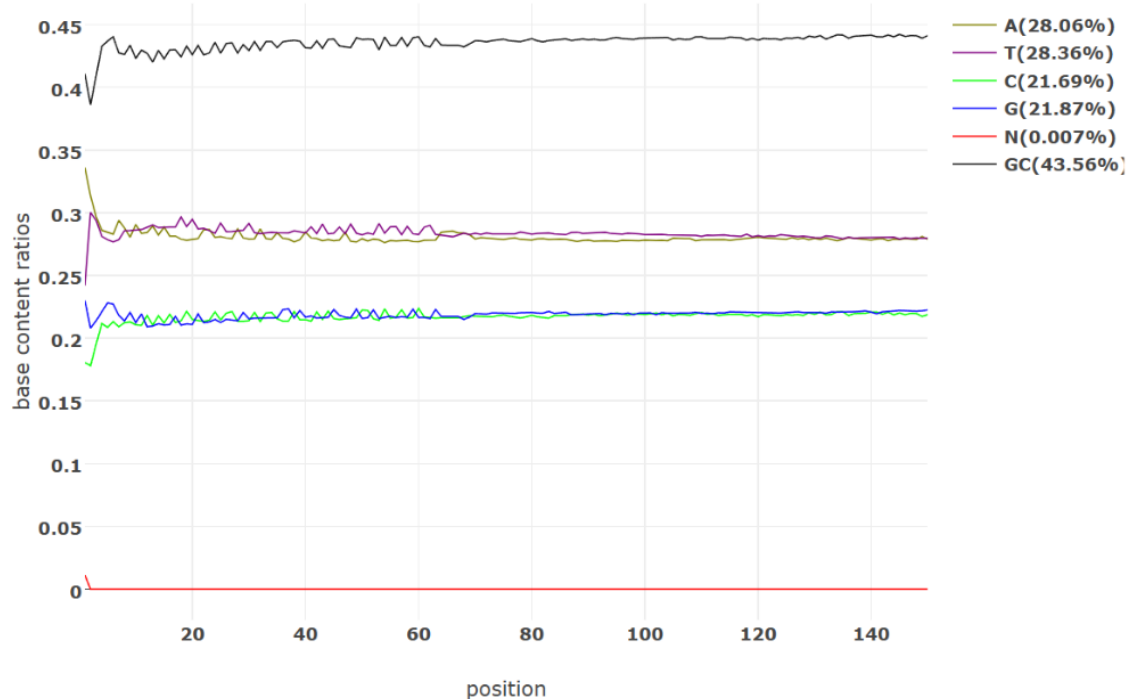


Figure 23: fastp report for 'ERR11468775' before filtering

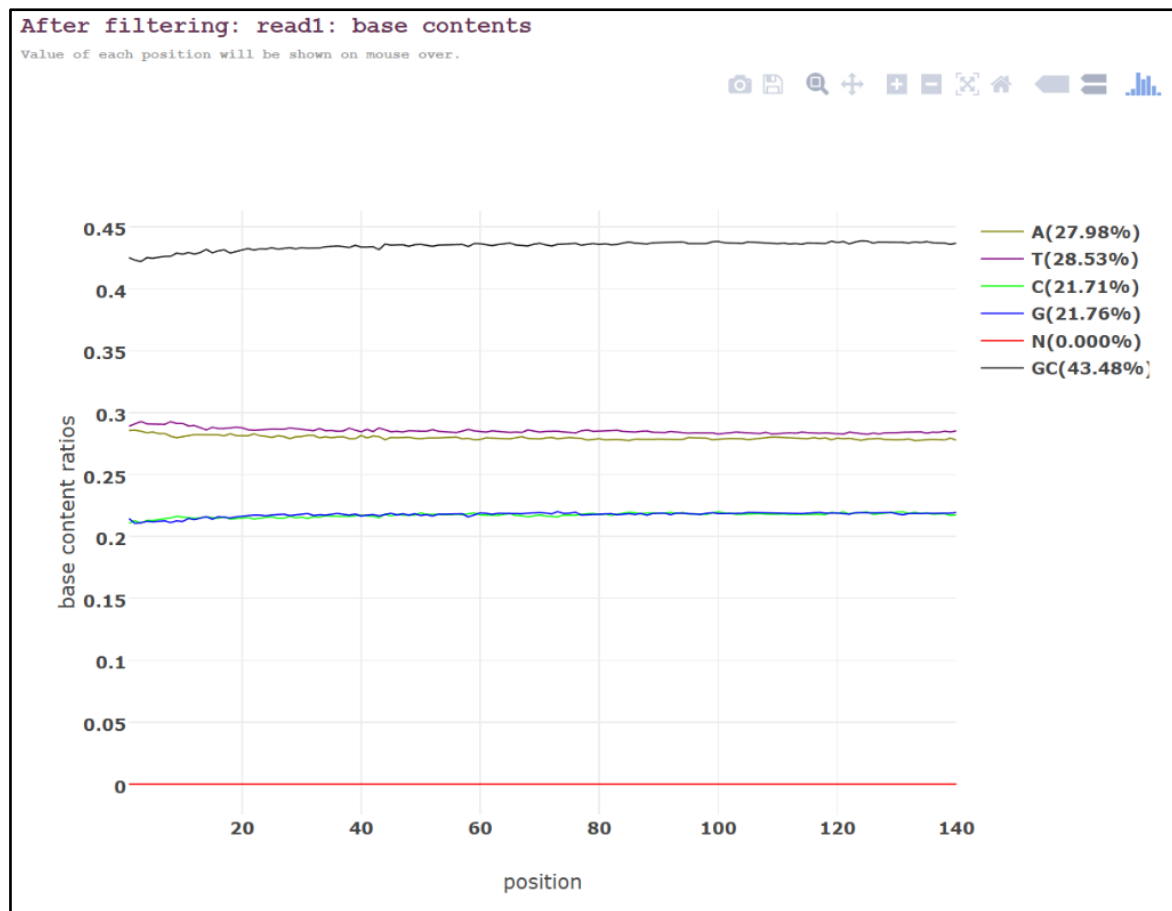


Figure 24: fastp report for ‘ERR11468775’ after filtering

NGS-Precision Medicine Toolkit
Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment SRA Downloader FastQC Fastp **Reference Genome** BWA Alignment GATK Analysis

Reference Genome Selection

Option 1: Upload Local File

Upload Reference Genome (FASTA/FASTQ)

Drop File Here
- OR -
Click to Upload

Custom Name (optional)
e.g., hg38_chr18

Option 2: Download from URL

Reference Genome URL
e.g., https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr18.fa.gz

Custom Name (optional)
e.g., hg38_chr18

Example URLs

Load Reference Genome

Reference Genome Log

Use via API Built with Gradio Settings

Figure 25: Homepage for ‘Reference Genome’ Tab

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment SRA Downloader FastQC Fastp **Reference Genome** BWA Alignment GATK Analysis

Reference Genome Selection

Option 1: Upload Local File

Upload Reference Genome (FASTA/FASTQ)

Drop File Here

- or -

Click to Upload

Custom Name (optional)

e.g., hg38_chr18

Option 2: Download from URL

Reference Genome URL

https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr18.fa.gz

Custom Name (optional)

e.g., hg38_chr18

Example URLs

UCSC hg38 chr18

Load Reference Genome

Reference Genome Log

Successfully downloaded reference genome

Use via API Built with Gradle Settings

Figure 26: Successfully downloaded reference genome

```

WSL: Ubuntu-22.04  main*  0 0 0 0  1 5.32
Ln 984, Col 18  Spaces: 4  UTF-8  LF  Python  3.10.12 64-bit  Go Live

reads corrected by overlap analysis: 22
bases corrected by overlap analysis: 48

Duplication rate: 0%

Insert size peak (evaluated by paired-end reads): 223

JSON report: fastp.json
HTML report: data/ERR11468775/fastp_reports/fastp_report.html

applications/fastp/fastp -i data/ERR11468775/ERR11468775_1.fastq.gz -o data/ERR11468775/ERR11468775_1_trimmed.fastq.gz -I data/ERR11468775/ERR11468775_2.fastq.gz -O data/ERR11468775/ERR11468775_2_trimmed.fastq.gz --detect_adapter_for_pe -f 10 -g -l 50 -c -h data/ERR11468775/fastp_reports/fastp_report.html -w 10
fastp v0.24.0, time used: 1 seconds

Downloading reference genome from https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr18.fa.gz...
--2025-04-03 06:15:14-- https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr18.fa.gz
Resolving hgdownload.soe.ucsc.edu (hgdownload.soe.ucsc.edu)... 128.114.119.163
Connecting to hgdownload.soe.ucsc.edu (hgdownload.soe.ucsc.edu)|128.114.119.163|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 25154367 (24M) [application/x-gzip]
Saving to: 'data/ERR11468775/reference/chr18.fa.gz'

data/ERR11468775/reference/chr18. 100%[=====] 23.99M 748KB/s in 39s

2025-04-03 06:16:02 (629 KB/s) - 'data/ERR11468775/reference/chr18.fa.gz' saved [25154367/25154367]

Decompressing gzip file...
Indexing reference genome...
[bwa_index] Pack FASTA... 0.70 sec
[bwa_index] Construct BWT for the packed sequence...
[bwtIncCreate] textlength=160746570, availableWord=23310700
[bwtIncConstructFromPacked] 10 iterations done. 38451706 characters processed.
[bwtIncConstructFromPacked] 20 iterations done. 71035482 characters processed.
[bwtIncConstructFromPacked] 30 iterations done. 99991994 characters processed.
[bwtIncConstructFromPacked] 40 iterations done. 125724586 characters processed.
[bwtIncConstructFromPacked] 50 iterations done. 148591754 characters processed.
[bwt_gen] Finished constructing BWT in 56 iterations.
[bwa_index] 35.85 seconds elapsed.
[bwa_index] Update BWT... 0.27 sec
[bwa_index] Pack forward-only FASTA... 0.39 sec
[bwa_index] Construct SA from BWT and Occ... 21.69 sec
[main] Version: 0.7.17-r1188
[main] CHD: bwa index data/ERR11468775/reference/chr18.fa
[main] Real time: 65.382 sec; CPU: 58.894 sec
  
```

Figure 27: Backend code running for downloading and processing the reference genome

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment

SRA Downloader

FastQC

Fastp

Reference Genome

BWA Alignment

GATK Analysis

BWA Alignment

Upload Reads File (FASTQ)

Drop File Here

- OR -

Click to Upload

Select Reference Genome

Run BWA Alignment

BWA Log

Aligned Output (BAM)

Alignment Algorithm

mem

aln

bwasw

Number of Threads

4

32

Figure 28: Homepage for ‘BWA Alignment’ Tab

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment

SRA Downloader

FastQC

Fastp

Reference Genome

BWA Alignment

GATK Analysis

BWA Alignment

Upload Reads File (FASTQ)

ERR11468775_1_trimmed.fastq.gz

9.6 KB

×

ERR11468775_2_trimmed.fastq.gz

9.8 KB

×

Select Reference Genome

Run BWA Alignment

BWA Log

Processed 190 reads in 0.053 CPU sec, 0.020 real sec

Aligned Output (BAM)

Retrieve the SAM file from ‘data/ERR11468775/alignment/aligned.sam’

Alignment Algorithm

mem

aln

bwasw

Number of Threads

4

32

Use via API

Built with Gradio

Settings

Figure 29: Processed the trimmed files and aligned using BWA Alignment Tool and MEM Algorithm

```

[BTIncCreate] textLength=169746576, availableWord=23310700
[BTIncConstructFromPacked] 10 iterations done. 38451706 characters processed.
[BTIncConstructFromPacked] 20 iterations done. 71035482 characters processed.
[BTIncConstructFromPacked] 30 iterations done. 99991994 characters processed.
[BTIncConstructFromPacked] 40 iterations done. 125724586 characters processed.
[BTIncConstructFromPacked] 50 iterations done. 148591754 characters processed.
[bwt_gen] Finished constructing BWT in 56 iterations.
[bwa_index] 35.85 seconds elapse.
[bwa_index] Update BWT... 0.27 sec
[bwa_index] Pack forward-only FASTA... 0.39 sec
[bwa_index] Construct SA from BWT and Occ... 21.69 sec
[main] Version: 0.7.17-r1188
[main] CMD: bwa index data/ERR11468775/reference/chr18.fa
[main] Real time: 65.382 sec; CPU: 58.894 sec
Indexing reference genome...
[bwa_index] Pack FASTA... 0.70 sec
[bwa_index] Construct BWT for the packed sequence...
[BTIncCreate] textLength=169746576, availableWord=23310700
[BTIncConstructFromPacked] 10 iterations done. 38451706 characters processed.
[BTIncConstructFromPacked] 20 iterations done. 71035482 characters processed.
[BTIncConstructFromPacked] 30 iterations done. 99991994 characters processed.
[BTIncConstructFromPacked] 40 iterations done. 125724586 characters processed.
[BTIncConstructFromPacked] 50 iterations done. 148591754 characters processed.
[bwt_gen] Finished constructing BWT in 56 iterations.
[bwa_index] 37.61 seconds elapse.
[bwa_index] Update BWT... 0.27 sec
[bwa_index] Pack forward-only FASTA... 0.38 sec
[bwa_index] Construct SA from BWT and Occ... 23.14 sec
[main] Version: 0.7.17-r1188
[main] CMD: bwa index -a bwtsw -p data/ERR11468775/bwa_index_map/chr18_ref data/ERR11468775/reference/chr18.fa
[main] Real time: 68.432 sec; CPU: 62.118 sec
Running BWA command: bwa mem data/ERR11468775/bwa_index_map/chr18_ref data/ERR11468775/1_trimmed.fastq.gz data/ERR11468775/2_trimmed.fastq.gz -t 4 -o data/ERR11468775/alignment/aligned.sam
[M::bwa_idx_load_from_disk] read 0 ALT contigs
[M::process] read 190 sequences (26430 bp)...
[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 5, 0, 0)
[M::mem_pestat] skip orientation FF as there are not enough pairs
[M::mem_pestat] skip orientation FR as there are not enough pairs
[M::mem_pestat] skip orientation RF as there are not enough pairs
[M::mem_pestat] skip orientation RR as there are not enough pairs
[M::mem_process_seqs] Processed 190 reads in 0.053 CPU sec, 0.020 real sec
[main] Version: 0.7.17-r1188
[main] CMD: bwa mem -t 4 -o data/ERR11468775/alignment/aligned.sam data/ERR11468775/bwa_index_map/chr18_ref data/ERR11468775/1_trimmed.fastq.gz data/ERR11468775/2_trimmed.fastq.gz
[main] Real time: 0.767 sec; CPU: 0.109 sec

```

Figure 30: Backend code running for aligning the trimmed files using the BWA tool

```

@SQ SN:chr18 LN:80373285
@PG ID:bwa PN:bwa VN:0.7.17-r1188 CL:bwa mem data/ERR11468775/bwa_index_map/chr18_ref data/ERR11468775/1_trimmed.f
ERR11468775.2 97 chr18 79703386 60 140H = 79703349 189 AAAATGTAACTGAGCTTAAGTCACTCTACCTTTTAAAGCCATTAAAGTAAT
ERR11468775.2 145 chr18 79703349 60 140H = 79703308 189 AGTAATATTTTACTGAGAAATAACAGGATTTAAAGGATCGAGTTTAACT
ERR11468775.3 97 chr18 33245893 60 140H = 33245927 174 GCATTGTAGTAAGATTTTTCAGCATCTCCACATAGATCCAGCAACATGCT
ERR11468775.3 145 chr18 33245927 60 140H = 33245893 174 CTAGATGCCACATCACTGCTCAAGTGTGTACBACAAATAAAGTCTCCAGCC
ERR11468775.4 77 * 0 0 * * 0 CTTATTTTAATAGCATATGCCCTAAAGTTTCAGCATGTTATAGCATATGACATAATTTCTTTTTTAAAGCCGAGATA
ERR11468775.4 141 * 0 0 * * 0 TTAGAAACAGAGTACCATATAATCCAGCAATCCACACCTGTGTATATATCCAAATAATTTTTTAAGTTTTCGTA
ERR11468775.5 77 * 0 0 * * 0 AGTCGATATTCTTCTGCACTACTGAGTCATGTTTAAAGTAATGAATAAATAATTTCTTCAATTTGAATAATAGTGTCTG
ERR11468775.5 141 * 0 0 * * 0 AAAAGATCCAAATCTTTTAAATCAAAATGAATCTTCTCATGATGGAATTTTAAAGACTCACCATTATTTGGAGACTTG
ERR11468775.6 77 * 0 0 * * 0 AAATTTGATTTCCAGATTGAAATATATATATTTAAATGTCAAGTTTCAACAAAATGCAAGATATACAAAGAACAC
ERR11468775.6 141 * 0 0 * * 0 CTGTGTTAGTCTAGTATATCCACCATAGGACACTCACTGGCTGTTCTGTTGACGCTGTTGTGTCGCAATAGGAC
ERR11468775.7 77 * 0 0 * * 0 TTACAGCTCTCTGAGTGAATAATCCAGCAGGTTCTTTCACATTGGCCACACTATGTTGTCGCAAGTCCCAATTTTC
ERR11468775.7 141 * 0 0 * * 0 CCAATACACAAATCTTCACTCTGCTGTGAAGATCTGTAACCAATAATTTGAAGATTAACTGCACCTTTATTTT
ERR11468775.8 73 chr18 3558754 27 5586M49S = 3558754 0 ATGCATCTTTATTTTGAAGTGGGTTCTTTTAGACTACTTAATTTGGGCTCTCTTTT
ERR11468775.8 133 chr18 3558754 0 * = 3558754 0 TACATACTAAGAAAGTAGAGAAATGGAATTCATAAAATATGAGCTTAAATACAAAGGACAGAAAA
ERR11468775.9 77 * 0 0 * * 0 GAATTCCTAGAGGCCACAGAGATCCAGGCAAGGCATTCTGAGGAAGCCTCTGCACACAGCTGAAACACAGGCGAGGCA
ERR11468775.9 141 * 0 0 * * 0 GTGTCTTTGGCATTCTTACCTTGGCGTTCTTGGCCAGGAGCTTGACACATATTCTCACTTGAAGATGCATTCAC
ERR11468775.10 113 chr18 41424729 18 165124M = 58071840 16647861 AAAGGAGTTTCTCCCATCTGTTCTCATGATAGTGAATAGTCTCAT
ERR11468775.10 177 chr18 58071840 0 28573M47S = 41424729 -16647861 GTGAAAGGGATACCCCTGTGAAACATCAATCTCATGAGAC
ERR11468775.11 77 * 0 0 * * 0 GGATGCCATTAGCCCAATAGACCTGATCACTTCACACCAAAATTTGTCACCATGATTTGGCAAGTTGAGAATTTAAG
ERR11468775.11 141 * 0 0 * * 0 GAAATTTTGTCTCAGAACATGCTGGCTTTTCAACATCATCCAGGCCAATATCTGATATTTTATTTACTCACTCACTG
ERR11468775.12 77 * 0 0 * * 0 CAGCCGGAGCCGCGAGGCTCTCGCGGGTTGTTGAGTCTCTGTTTAAAGTGGGAGGCTCTCGCGGCTCTCAGCGAGGCTACC
ERR11468775.12 141 * 0 0 * * 0 CGCTCGAATACACAGCACCGCCAGCGTGGACCTCTGAGCGCGGATACATCCACACCCAGAGCCGACCGGATC
ERR11468775.13 77 * 0 0 * * 0 CCCCACCCAGGAGCGAGGAGGAGGAGGCTGCGACCTCTGCTCTCCACCGCTCAGCGCGGCGCTTGGGCTCTG

```

Figure 31: Display of aligned.sam file

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment

SRA Downloader

FastQC

Fastp

Reference Genome

BWA Alignment

GATK Analysis

GATK Variant Analysis

Upload Aligned Reads (BAM)

Drop File Here
- OR -
Click to Upload

Select Reference Genome

Analysis Mode

Full

Germline

Somatic

☐ Skip downloading resource files (use existing)

Run GATK Analysis

GATK Log

Figure 32: Homepage for ‘GATK Analysis’ Tab

NGS-Precision Medicine Toolkit

Streamlining NGS workflows for Applications of Precision Medicine

Setup Environment

SRA Downloader

FastQC

Fastp

Reference Genome

BWA Alignment

GATK Analysis

GATK Variant Analysis

Upload Aligned Reads (BAM)

aligned.sam 68.3 KB

Select Reference Genome

Analysis Mode

Full

Germline

Somatic

☐ Skip downloading resource files (use existing)

Run GATK Analysis

GATK Log

Running samtools...
Running GATK Analysis...
Completed GATK Analysis.
Retrieve the variant call file at 'data/ERR11468775/SBGATK_ANALYSIS/try_new.vcf.gz'

Figure 33: Processing Variant files using GATK Docker

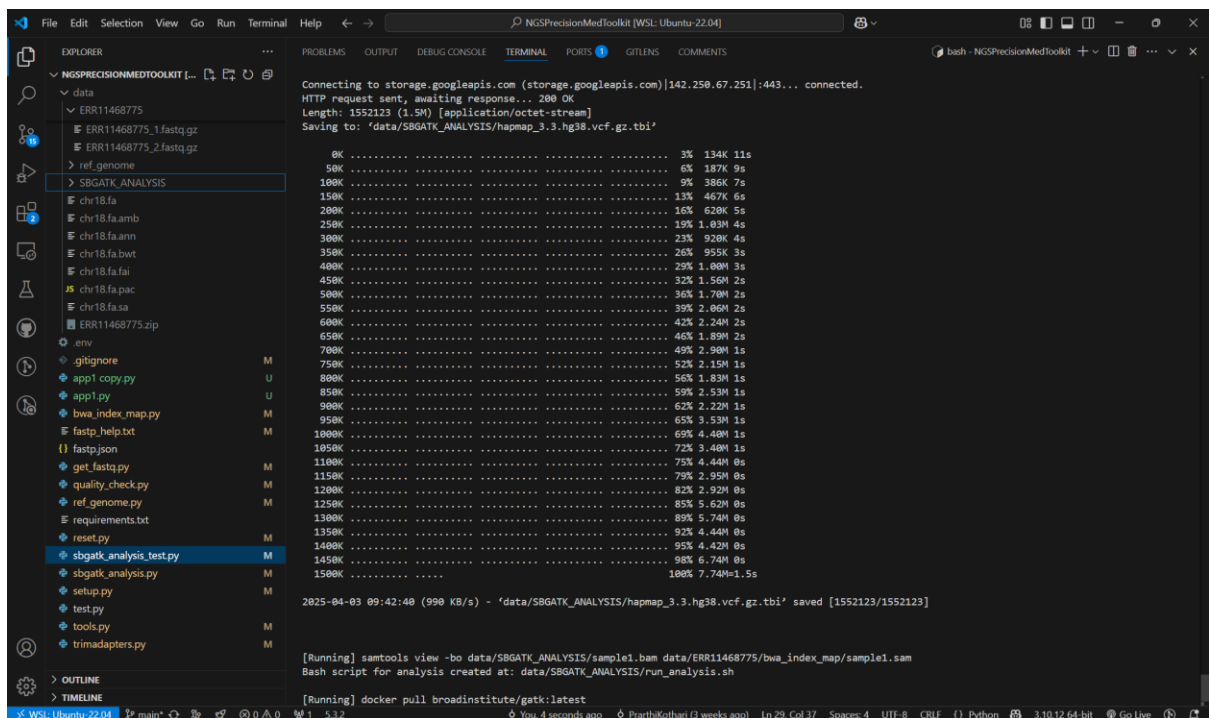


Figure 34: Backend code running for generating variant calling files using GATK tool within the docker container

CH86	A	B	C	D	E	F	G	H	I	J	K	L	M
	Uploaded_variation	Location	Allele	Consequence	IMPACT	SYMBOL	Gene	Feature_type	Feature	BIOTYPE	EXON	INTRON	HGVSc
1		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000577719.5	lncRNA	-	01-Mar	ENST00000577719.5.m.308-61892G>C
2		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000579835.5	lncRNA	-	02-Apr	ENST00000579835.5.m.272-10416G>C
3		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000580781.5	lncRNA	-	01-Apr	ENST00000580781.5.m.307-61892G>C
4		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753289.1	lncRNA	-	01-Jun	ENST00000753289.1.m.154-61892G>C
5		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753290.1	lncRNA	-	01-May	ENST00000753290.1.m.229-87228G>C
6		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753291.1	lncRNA	-	01-Jun	ENST00000753291.1.m.41-61885C>T
7		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753292.1	lncRNA	-	01-May	ENST00000753292.1.m.129-87228G>C
8		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753293.1	lncRNA	-	01-Apr	ENST00000753293.1.m.267-87228G>C
9		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753294.1	lncRNA	-	01-Jun	ENST00000753294.1.m.157-61892G>C
10		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753295.1	lncRNA	-	01-Feb	ENST00000753295.1.m.238-87228G>C
11		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753296.1	lncRNA	-	01-Apr	ENST00000753296.1.m.40-61892G>C
12		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753297.1	lncRNA	-	01-Jun	ENST00000753297.1.m.118-61892G>C
13		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753298.1	lncRNA	-	01-Mar	ENST00000753298.1.m.203-61888A>G
14		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753299.1	lncRNA	-	01-Feb	ENST00000753299.1.m.207-87228G>C
15		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753299.1	lncRNA	-	01-Mar	ENST00000753299.1.m.308-61888A>G
16		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000577719.5	lncRNA	-	02-Apr	ENST00000577719.5.m.272-10420A>G
17		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000579835.5	lncRNA	-	01-Apr	ENST00000579835.5.m.272-10420A>G
18		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000580781.5	lncRNA	-	01-Apr	ENST00000580781.5.m.307-61888A>G
19		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753289.1	lncRNA	-	01-Jun	ENST00000753289.1.m.154-61888A>G
20		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753290.1	lncRNA	-	01-May	ENST00000753290.1.m.229-87224A>G
21		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753291.1	lncRNA	-	01-Jun	ENST00000753291.1.m.41-61888A>G
22		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753292.1	lncRNA	-	01-May	ENST00000753292.1.m.129-87224A>G
23		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753293.1	lncRNA	-	01-Apr	ENST00000753293.1.m.267-87224A>G
24		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753294.1	lncRNA	-	01-Jun	ENST00000753294.1.m.157-61888A>G
25		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753295.1	lncRNA	-	01-Feb	ENST00000753295.1.m.238-87224A>G
26		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753296.1	lncRNA	-	01-Apr	ENST00000753296.1.m.40-61888A>G
27		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753297.1	lncRNA	-	01-May	ENST00000753297.1.m.118-61888A>G
28		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753298.1	lncRNA	-	01-Mar	ENST00000753298.1.m.203-61888A>G
29		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753299.1	lncRNA	-	01-Feb	ENST00000753299.1.m.207-87224A>G
30		18:1071951-1071951	C	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000577719.5	lncRNA	-	01-Mar	ENST00000577719.5.m.308-61885C>T
31		18:1071958-1071958	T	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000579835.5	lncRNA	-	02-Apr	ENST00000579835.5.m.272-10423C>T
32		18:1071958-1071958	T	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000580781.5	lncRNA	-	01-Apr	ENST00000580781.5.m.307-61885C>T
33		18:1071958-1071958	T	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753289.1	lncRNA	-	01-Jun	ENST00000753289.1.m.154-61885C>T
34		18:1071958-1071958	T	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753290.1	lncRNA	-	01-May	ENST00000753290.1.m.229-87221C>T
35		18:1071958-1071958	T	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753291.1	lncRNA	-	01-Jun	ENST00000753291.1.m.41-61885C>T
36		18:1071958-1071958	T	intron_variant,non_coding_transcript_variant	MODIFIER-		ENSG00000263551	Transcript	ENST00000753292.1	lncRNA	-	01-May	ENST00000753292.1.m.129-87221C>T

Figure 35: Variant Effects predicted using the generated variant calling file

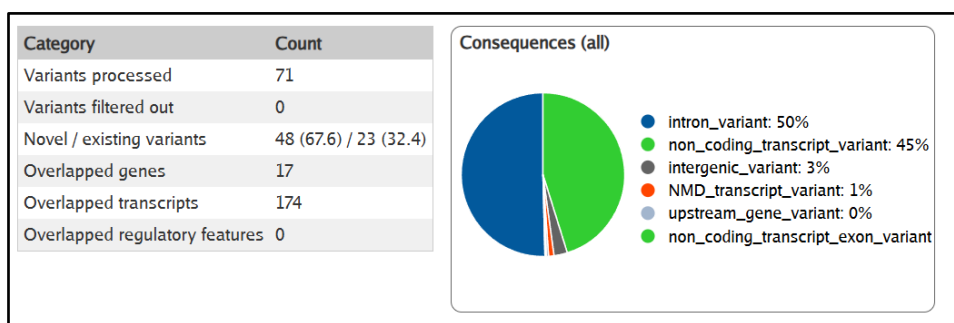


Figure 36: Statistics of Variant Effects

CHAPTER 8: DISCUSSION

DISCUSSION

Despite its promise, precision medicine faces several challenges. Critics argue that many of its anticipated benefits remain unfulfilled, particularly concerning complex diseases where genetic factors alone do not dictate treatment responses. Additionally, the high costs associated with new biotechnologies may exacerbate health inequalities and pose sustainability issues for healthcare systems, especially in low- and middle-income countries.

Moving forward, continued investment in research is essential to overcome these barriers. There is a need for improved diagnostic methods and treatments that are accessible and affordable for diverse populations. As precision medicine evolves, it holds the potential not only for enhanced individual care but also for broader public health improvements through more effective disease management strategies.

As NGS technology continues to evolve, several future directions and challenges emerge:

- 1. Integration with Clinical Practice:** The integration of NGS into routine clinical practice remains a challenge due to regulatory hurdles and the need for standardized protocols. Ensuring that genomic data is interpreted correctly and used effectively in clinical decision-making is crucial for maximizing its benefits.
- 2. Ethical Considerations:** The implications of genetic information raise ethical concerns regarding privacy, consent, and potential discrimination based on genetic predispositions. Addressing these issues will be essential as NGS becomes more prevalent in both research and healthcare settings.
- 3. Data Interpretation:** While data generation has become more efficient, interpreting the vast amounts of genomic data remains complex. Continued advancements in bioinformatics tools and algorithms are necessary to translate genomic findings into actionable insights effectively.

As technology continues to evolve, the role of GUIs in NGS workflows will likely expand further:

- 1. Integration with Artificial Intelligence:** Future GUIs may incorporate AI-driven analytics that assist users in interpreting complex genomic data more effectively.
- 2. Mobile Accessibility:** With the rise of mobile technology, developing mobile-friendly GUIs could allow researchers to access NGS tools on-the-go, increasing flexibility in data analysis.
- 3. Enhanced Visualization Techniques:** Continued advancements in visualization techniques will improve how genomic data is presented within GUIs, making it easier for users to understand intricate patterns and relationships within the data.

The detailed methodology outlines a structured approach to developing an integrated NGS workflow with a user-friendly GUI. By systematically addressing each phase—from problem identification and literature review to pipeline design, backend integration, validation, GUI development, and final release—the project ensures the creation of a robust, efficient, and accessible tool for precision medicine applications. Comprehensive documentation and

iterative user feedback further guarantee that the final product meets the needs of its target users, fostering advancements in genomic research and personalized healthcare.

In summary, Graphical User Interfaces are vital for enhancing the usability and efficiency of NGS workflows. Their implementation not only streamlines processes but also supports advancements in precision medicine by making genomic research more accessible and collaborative. As technology progresses, GUIs will continue to play a crucial role in shaping the future of genomic analysis and personalized healthcare solutions.

CHAPTER 9: CONCLUSION

CONCLUSION

The integration of NGS into precision medicine has already demonstrated substantial benefits in improving treatment outcomes, particularly in oncology. Studies have shown that NGS-guided therapies increase response rates and progression-free survival, reinforcing the value of genomic profiling in personalized treatment strategies. However, significant challenges—including data standardization, ethical considerations, high costs, and clinical integration barriers—must be addressed to maximize NGS’s full potential. Emerging technologies, such as nanopore sequencing and AI-driven genomic analysis, are poised to further revolutionize the field by improving data processing efficiency, reducing costs, and expanding accessibility. Additionally, the continued advancement of GUIs will play a pivotal role in simplifying genomic data interpretation, ensuring that both researchers and clinicians can effectively utilize NGS for precision medicine applications. As NGS technology continues to evolve, sustained investment in research, policy development, and innovative software solutions will be essential in driving precision medicine forward. By overcoming existing barriers and fostering widespread adoption, NGS has the potential to transform healthcare, making personalized treatment strategies a standard practice in modern medicine.

CHAPTER 10: REFERENCES

REFERENCES

1. Andrews, S. (2010). FastQC: A Quality Control Tool for High Throughput Sequence Data [Online]. Available online at: <http://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
2. Appiah, Vincent. (2021). Download Fastq Data using SRA Toolkit. https://www.researchgate.net/publication/355651665_Download_Fastq_Data_using_SRA_Toolkit
3. Babraham Bioinformatics - FastQC A Quality Control tool for High Throughput Sequence Data. (n.d.). <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
4. Chen, S., Zhou, Y., Chen, Y., & Gu, J. (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics* (Oxford, England), 34(17), i884–i890. <https://doi.org/10.1093/bioinformatics/bty560>
5. Ewels, P., Magnusson, M., Lundin, S., & Källér, M. (2016). MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics* (Oxford, England), 32(19), 3047–3048. <https://doi.org/10.1093/bioinformatics/btw354>
6. Jameson, J. L., & Longo, D. L. (2015). Precision medicine--personalized, problematic, and promising. *The New England journal of medicine*, 372(23), 2229–2234. <https://doi.org/10.1056/NEJMsbl503104>
7. Khetani, M. P. R. (2018, September 5). Quality control: Assessing FASTQC results. Introduction to RNA-Seq using high-performance computing - ARCHIVED. https://hbctraining.github.io/Intro-to-rnaseq-hpc-salmon/lessons/qc_fastqc_assessment.html
8. Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics* (Oxford, England), 25(16), 2078–2079. <https://doi.org/10.1093/bioinformatics/btp352>
9. McLaren, W., Gil, L., Hunt, S. E., Riat, H. S., Ritchie, G. R., Thormann, A., Flicek, P., & Cunningham, F. (2016). The Ensembl Variant Effect Predictor. *Genome biology*, 17(1), 122. <https://doi.org/10.1186/s13059-016-0974-4>
10. Morash, M., Mitchell, H., Beltran, H., Elemento, O., & Pathak, J. (2018). The Role of Next-Generation Sequencing in Precision Medicine: A Review of Outcomes in Oncology. *Journal of personalized medicine*, 8(3), 30. <https://doi.org/10.3390/jpm8030030>
11. Morganti, S., Tarantino, P., Ferraro, E., D'Amico, P., Duso, B. A., & Curigliano, G. (2019). Next Generation Sequencing (NGS): A Revolutionary Technology in Pharmacogenomics and Personalized Medicine in Cancer. *Advances in experimental medicine and biology*, 1168, 9–30. https://doi.org/10.1007/978-3-030-24100-1_2
12. Mosele, M. F., Westphalen, C. B., Stenzinger, A., Barlesi, F., Bayle, A., Bièche, I., Bonastre, J., Castro, E., Dienstmann, R., Krämer, A., Czarnecka, A. M., Meric-Bernstam, F., Michiels, S., Miller, R., Normanno, N., Reis-Filho, J., Remon, J., Robson, M., Rouleau, E., Scarpa, A., ... André, F. (2024). Recommendations for the use of next-generation sequencing (NGS) for patients with advanced cancer in 2024: a report from the ESMO Precision Medicine Working Group. *Annals of oncology : official journal of the European Society for Medical Oncology*, 35(7), 588–606. <https://doi.org/10.1016/j.annonc.2024.04.005>

13. Ncbi. (n.d.). Home. GitHub. <https://github.com/ncbi/sra-tools/wiki>
 14. Ren, S., Ahmed, N., Bertels, K., & Al-Ars, Z. (2019). GPU accelerated sequence alignment with traceback for GATK HaplotypeCaller. BMC genomics, 20(Suppl 2), 184. <https://doi.org/10.1186/s12864-019-5468-9>
 15. Rimmer, A., Phan, H., Mathieson, I., Iqbal, Z., Twigg, S. R. F., WGS500 Consortium, Wilkie, A. O. M., McVean, G., & Lunter, G. (2014). Integrating mapping-, assembly- and haplotype-based approaches for calling variants in clinical sequencing applications. Nature genetics, 46(8), 912–918. <https://doi.org/10.1038/ng.3036>
 16. Schmid, S., Jochum, W., Padberg, B., Demmer, I., Mertz, K. D., Joerger, M., Britschgi, C., Matter, M. S., Rothschild, S. I., & Omlin, A. (2022). How to read a next-generation sequencing report-what oncologists need to know. ESMO open, 7(5), 100570. <https://doi.org/10.1016/j.esmoop.2022.100570>
 17. Tenenbaum, J. D., Avillach, P., Benham-Hutchins, M., Breitenstein, M. K., Crowgey, E. L., Hoffman, M. A., Jiang, X., Madhavan, S., Mattison, J. E., Nagarajan, R., Ray, B., Shin, D., Visweswaran, S., Zhao, Z., & Freimuth, R. R. (2016). An informatics research agenda to support precision medicine: seven key areas. Journal of the American Medical Informatics Association : JAMIA, 23(4), 791–795. <https://doi.org/10.1093/jamia/ocv213>
 18. Yadav, D., Patil-Takbhate, B., Khandagale, A., Bhawalkar, J., Tripathy, S., & Khopkar-Kale, P. (2023). Next-Generation sequencing transforming clinical practice and precision medicine. Clinica chimica acta; international journal of clinical chemistry, 551, 117568. <https://doi.org/10.1016/j.cca.2023.117568>
-